

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐỒ ÁN 1

XÂY DỰNG VÀ TỐI ƯU MÔ HÌNH YOLO26
CHO THIẾT BỊ BIÊN

Building and Optimizing YOLO26 Model for Edge Devices

Giảng viên hướng dẫn: ThS. Trần Hoàng Lộc

Sinh viên thực hiện: Bùi Quốc Bảo — 23520092
Võ Nhật Khiêm — 23520731

TP. Hồ Chí Minh, 2026

Mục lục

1	Giới thiệu	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu đồ án	1
1.3	Phạm vi đồ án	2
1.4	Phương pháp nghiên cứu	2
1.5	Cấu trúc báo cáo	3
2	Cơ sở lý thuyết	4
2.1	Tổng quan Object Detection	4
2.1.1	Phân loại kiến trúc	4
2.1.2	Các chỉ số đánh giá	4
2.2	Dòng YOLO	5
2.2.1	Lịch sử phát triển	5
2.2.2	Đặc điểm nổi bật của YOLO26	5
2.3	Các khái niệm nền tảng	6
2.3.1	Tích chập (Convolution)	6
2.3.2	Batch Normalization (BN)	6
2.3.3	Hàm kích hoạt SiLU	6
2.3.4	Feature Pyramid Network (FPN) và PANet	6
2.3.5	CIoU (Complete Intersection over Union)	7
2.4	Edge Computing và Raspberry Pi 5	7
2.5	NCNN Framework	7
3	Xây dựng bộ dữ liệu	9
3.1	Phân tích yêu cầu dataset	9
3.2	Khảo sát và lựa chọn dataset	9
3.2.1	COCO 2017 (Common Objects in Context)	9
3.2.2	SCB-05 (Smart Classroom Behavior)	9
3.2.3	TED Talk (tự tạo)	10
3.2.4	Kết luận lựa chọn	10
3.3	Dataset COCO Person 320	10
3.3.1	Quy trình tiền xử lý	10
3.3.2	Thống kê dataset	10
3.3.3	Phân tích dữ liệu khám phá (EDA)	11
3.4	Định dạng YOLO	12
3.5	Hướng mở rộng dataset	12

4	Kiến trúc mô hình YOLO26	13
4.1	Backbone	13
4.1.1	Khối Conv cơ bản	13
4.1.2	Khối DWConv (Depthwise Convolution)	14
4.1.3	Khối Bottleneck	14
4.1.4	Khối C2f (Cross Stage Partial with 2 Convolutions)	15
4.1.5	Khối C3 và C3k	15
4.1.6	Khối C3k2	16
4.1.7	Khối SPPF (Spatial Pyramid Pooling Fast)	17
4.1.8	Cơ chế Attention (Multi-Head Self-Attention)	18
4.1.9	Khối PSABlock (Position-Sensitive Attention Block)	19
4.1.10	Khối C2PSA (Channel-Split Position-Sensitive Attention)	19
4.1.11	Tổng quan đồ thị các khối đặc biệt	20
4.1.12	Bảng chi tiết các lớp Backbone	21
4.2	Neck	21
4.2.1	Top-Down Pathway (FPN)	21
4.2.2	Bottom-Up Pathway (PAN)	21
4.3	Detection Head: Dual-Head O2M/O2O	22
4.3.1	Nhánh cv2 — Box Prediction	22
4.3.2	Nhánh cv3 — Class Prediction	23
4.3.3	Cơ chế Dual-Head	23
4.4	Box Decoding: Direct Regression	23
4.4.1	Cơ chế DFL (Distribution Focal Loss)	24
4.4.2	So sánh Direct Regression và DFL	24
4.5	Phân tỷ lệ mô hình (Model Scaling)	25
5	Huấn luyện và tối ưu mô hình	26
5.1	Huấn luyện YOLO26 gốc (Ultralytics)	26
5.1.1	Thiết lập	26
5.1.2	Kết quả	27
5.2	Custom YOLO26 from scratch	28
5.2.1	Động lực	28
5.2.2	Các tùy biến so với gốc	28
5.3	Hàm mất mát (Loss Functions)	28
5.4	TaskAlignedAssigner (TAL)	29
5.5	ProgLoss (Progressive Loss Balancing)	30
5.5.1	Công thức Loss tổng hợp đầy đủ	30
5.6	MuSGD Optimizer	31
5.7	STAL (Small-Target-Aware Label Assignment)	33
5.8	Hàm strip_o2m()	34
5.9	Kết quả Custom (1st run)	34

5.10	Export pipeline	35
6	Triển khai trên Raspberry Pi 5	36
6.1	Phần cứng	36
6.2	Phần mềm và kết nối	36
6.3	Benchmark FPS	36
6.4	So sánh với nghiên cứu tiền nhiệm	38
6.5	Demo	38
7	Kết quả và đánh giá	39
7.1	Tổng hợp kết quả huấn luyện	39
7.2	Kết quả inference trên Raspberry Pi 5	40
7.3	Đánh giá	40
7.3.1	Thành tựu	40
7.3.2	Hạn chế	40
7.4	Hướng phát triển	41
7.4.1	Đồ án 2 (dự kiến)	41
7.4.2	Khóa luận tốt nghiệp (dự kiến)	41
A	Phụ lục: Code nguồn chính	42
A.1	Kiến trúc YOLO26n	42
A.2	make_anchors	42
A.3	dist2bbox và bbox2dist	43
A.4	Newton–Schulz Iteration	43
A.5	E2ELoss decay	44
	Tài liệu tham khảo	45

Danh sách hình vẽ

3.1	Phân phối số lượng đối tượng person trên mỗi ảnh trong tập COCO Person 320	11
3.2	(Trái) Heatmap tọa độ tâm bounding box. (Phải) Phân phối tỷ lệ cao/rộng của bounding box.	11
4.1	Sơ đồ khối tổng quát kiến trúc YOLO26. Đầu vào ảnh $3 \times 640 \times 640$ được xử lý qua Backbone (trích xuất đặc trưng), Neck (tổng hợp FPN+PAN), và Detection Head (Dual-Head O2M/O2O).	13
4.2	Sơ đồ khối Bottleneck với skip connection.	14
4.3	Sơ đồ khối C3k — C3 với kernel 3x3 sâu hơn.	15
4.4	Sơ đồ khối C3k2 — forward theo C2f, block bên trong tùy cấu hình.	16
4.5	Sơ đồ khối SPPF — MaxPool liên tiếp tạo multi-scale receptive field.	17
4.6	Mình họa sự mở rộng receptive field tương đương 5×5 , 9×9 , và 13×13 qua 3 lần MaxPool 5x5 liên tiếp.	17
4.7	Sơ đồ khối Attention với QKV projection và Position Encoding.	18
4.8	Chi tiết luồng xử lý và sự biến đổi kích thước tensor trong module Attention. . .	19
4.9	Sơ đồ PSABlock: Attention + FFN với các kết nối residual.	19
4.10	Sơ đồ khối FFN (Feed-Forward Network) trong PSABlock.	19
4.11	Sơ đồ C2PSA: CSP splitting + PSABlock.	20
4.12	Mô phỏng sự biến đổi của feature map khi đi qua SPPF và C2PSA.	20
4.13	Tổng quan cách tổ chức các khối đặc biệt: C3k2, SPPF, và C2PSA trong YOLO26.	20
5.1	Biểu đồ loss curves trong quá trình huấn luyện YOLO26 gốc qua 120 epochs. .	27
5.2	Ma trận nhầm lẫn (Confusion Matrix) của YOLO26 gốc trên tập validation. . .	28
5.3	Biểu đồ thể hiện sự biến đổi tỷ lệ của hai nhánh O2M (α) và O2O (β) theo tiến trình huấn luyện (ProgLoss).	30
5.4	So sánh quy trình cập nhật trọng số giữa SGD và Muon trong bộ tối ưu MuSGD.	31
5.5	Tốc độ hội tụ trực giao hóa của thuật toán lặp Newton-Schulz so với SVD. . . .	32
6.1	So sánh FPS giữa YOLO26 NCNN và YOLOv8 NCNN trên Raspberry Pi 5 ở các độ phân giải đầu vào 320, 480, 640.	37
6.2	Demo nhận diện người trên Raspberry Pi 5 với camera 1280x720, input resolution 640, đạt 18.9 FPS. Detection: id:1 person 0.87.	38
7.1	Biểu đồ kết quả huấn luyện và so sánh mAP của mô hình YOLO26 Custom qua các epochs (trực quan hóa từ dữ liệu results.csv).	39
7.2	Hình ảnh mô phỏng kết quả nhận diện thực tế (prediction samples) của mô hình YOLO26.	40

Danh sách bảng

3.1	Thống kê bộ dữ liệu COCO Person 320	11
4.1	Sử dụng C3k2 tại các vị trí trong YOLO26.	16
4.2	Chi tiết các lớp trong Backbone của YOLO26n (scale nano, ảnh 320×320). . .	21
4.3	Số anchor points tại mỗi scale detection (ảnh 320×320).	22
4.4	So sánh nhánh O2M và O2O trong Dual-Head	23
4.5	So sánh model trước và sau khi strip O2M.	23
4.6	So sánh hồi quy trực tiếp (Direct Regression) và DFL	24
4.7	Cấu hình Scale của YOLO26.	25
5.1	Siêu tham số huấn luyện YOLO26 gốc	27
5.2	Kết quả huấn luyện YOLO26 gốc (epoch 120)	27
5.3	So sánh cấu hình YOLO26 gốc (Ultralytics) và custom	28
5.4	Trọng số các thành phần loss (thay đổi theo ProgLoss).	29
5.5	Lịch trình trọng số O2M/O2O qua epoch.	30
5.6	Kết quả huấn luyện YOLO26 Custom — lần chạy 1 (79 epochs)	34
6.1	Cấu hình phần cứng hệ thống	36
6.2	Benchmark FPS trên Raspberry Pi 5 (NCNN, detection-only, tắt GUI)	37
6.3	So sánh hệ thống triển khai với nghiên cứu tiền nhiệm [1]	38
7.1	Tổng hợp kết quả huấn luyện các phiên bản YOLO26	39

CHƯƠNG 1

Giới thiệu

1.1 Đặt vấn đề

Trong bối cảnh giáo dục và truyền thông hiện đại, nhu cầu ghi hình tự động các buổi thuyết trình, giảng dạy và hội thảo ngày càng gia tăng. Các hệ thống camera cố định truyền thống gặp nhiều hạn chế khi diễn giả di chuyển liên tục trong không gian thuyết trình, dẫn đến việc mất đối tượng khỏi khung hình hoặc cần sự can thiệp thủ công của người quay phim.

Giải pháp sử dụng camera xoay tự động bám theo diễn giả là một hướng tiếp cận hiệu quả, tuy nhiên đòi hỏi sự kết hợp chặt chẽ giữa thuật toán nhận diện đối tượng thời gian thực và hệ thống điều khiển cơ điện tử. Thách thức lớn nhất nằm ở việc triển khai các mô hình học sâu (deep learning) trên các thiết bị nhúng có tài nguyên hạn chế như Raspberry Pi, nơi mà tốc độ suy luận (inference speed) là yếu tố quyết định tính khả thi của toàn bộ hệ thống.

Các nghiên cứu tiền nhiệm tại Trường Đại học Công nghệ Thông tin [1] đã hiện thực thành công hệ thống camera bám vết diễn giả, tuy nhiên phải sử dụng bộ tăng tốc AI chuyên dụng Google Coral TPU để đạt tốc độ thời gian thực trên Raspberry Pi 5. Điều này làm tăng chi phí phần cứng và phức tạp hóa quy trình triển khai (pipeline chuyển đổi mô hình qua 4 bước: PyTorch → ONNX → TensorFlow → TFLite INT8 → Edge TPU).

Sự ra đời của YOLO26 [2] với các cải tiến kiến trúc quan trọng — loại bỏ hoàn toàn NMS (Non-Maximum Suppression), sử dụng hồi quy trực tiếp (Direct Regression) thay cho DFL (Distribution Focal Loss) — mở ra khả năng đạt hiệu năng thời gian thực trên CPU thuần của thiết bị nhúng mà không cần bất kỳ bộ tăng tốc phần cứng nào.

1.2 Mục tiêu đề án

Đề án 1 đặt ra các mục tiêu cụ thể sau:

- 1) **Nghiên cứu kiến trúc YOLO26:** Tìm hiểu sâu toàn bộ kiến trúc mô hình YOLO26 từ Backbone, Neck đến Detection Head, bao gồm các thành phần cốt lõi như C3k2, SPPF, C2PSA, Dual-Head (O2M/O2O), và cơ chế huấn luyện ProgLoss, MuSGD.
- 2) **Xây dựng và tiền xử lý dataset:** Lựa chọn, lọc và chuẩn bị bộ dữ liệu phù hợp cho bài toán nhận diện người (person detection) với kích thước đầu vào 320×320 pixel.
- 3) **Huấn luyện mô hình:** Tiến hành huấn luyện YOLO26 trên dataset đã chuẩn bị, bao gồm cả phiên bản sử dụng thư viện Ultralytics và phiên bản custom từ đầu (from scratch).
- 4) **Triển khai trên Raspberry Pi 5:** Chuyển đổi mô hình sang định dạng NCNN và triển khai demo nhận diện người trên Raspberry Pi 5 4GB mà không sử dụng bộ tăng tốc AI,

đánh giá hiệu năng FPS thực tế.

1.3 Phạm vi đồ án

Đồ án này là **giai đoạn 1** trong lộ trình 3 giai đoạn:

- **Đồ án 1** (hiện tại): Tập trung vào *detection* — nghiên cứu kiến trúc, huấn luyện mô hình và demo nhận diện người trên Raspberry Pi 5. Chưa tích hợp thuật toán bám vết (tracking) hay điều khiển servo.
- **Đồ án 2** (dự kiến): Tích hợp thuật toán bám vết ByteTrack [3], xây dựng pipeline detection–tracking hoàn chỉnh trên Raspberry Pi 5, bắt đầu thiết kế hệ thống điều khiển servo.
- **Khóa luận tốt nghiệp** (dự kiến): Hoàn thiện toàn bộ hệ thống camera tự động bám vết diễn giả, bao gồm điều khiển Pan-Tilt, truyền dẫn video thời gian thực, và giao diện giám sát.

Phạm vi cụ thể của Đồ án 1:

- **Đối tượng**: Diễn giả, giảng viên trong môi trường giảng đường, hội trường (không gặp vấn đề che khuất — occlusion).
- **Phần cứng**: Raspberry Pi 5 4GB, không có bộ tăng tốc AI (Google Coral, Hailo, v.v.).
- **Phần mềm**: Mô hình YOLO26 nano, runtime NCNN, hệ điều hành Raspberry Pi OS.
- **Kết nối**: SSH từ laptop Windows qua WiFi (hỗ trợ Tailscale cho kết nối xuyên router).

1.4 Phương pháp nghiên cứu

Đồ án sử dụng các phương pháp nghiên cứu sau:

- 1) **Nghiên cứu tài liệu**: Tham khảo paper gốc của YOLO26 [2], mã nguồn Ultralytics [4], và khóa luận tiền nhiệm [1] để nắm bắt kiến thức nền tảng.
- 2) **Thực nghiệm**: Huấn luyện mô hình trên nền tảng Kaggle (GPU), đánh giá kết quả bằng các chỉ số mAP50, mAP50-95, Precision, Recall.
- 3) **Custom implementation**: Xây dựng lại YOLO26 từ đầu (from scratch) nhằm hiểu sâu kiến trúc và hỗ trợ quá trình tùy biến mô hình.
- 4) **Đánh giá trên thiết bị thực**: Triển khai mô hình trên Raspberry Pi 5 và đo đạc FPS thực tế ở nhiều độ phân giải đầu vào khác nhau (320, 480, 640).

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1:** Giới thiệu tổng quan đề án, mục tiêu, phạm vi và phương pháp nghiên cứu.
- **Chương 2:** Trình bày cơ sở lý thuyết về Object Detection, dòng YOLO, Edge Computing và NCNN.
- **Chương 3:** Mô tả quá trình lựa chọn, xây dựng và phân tích bộ dữ liệu.
- **Chương 4:** Phân tích chi tiết kiến trúc mô hình YOLO26.
- **Chương 5:** Trình bày quá trình huấn luyện mô hình gốc và custom.
- **Chương 6:** Mô tả quá trình triển khai và đánh giá trên Raspberry Pi 5.
- **Chương 7:** Tổng hợp kết quả, đánh giá và hướng phát triển.

CHƯƠNG 2

Cơ sở lý thuyết

2.1 Tổng quan Object Detection

Object Detection (phát hiện đối tượng) là bài toán kết hợp hai tác vụ: *phân loại* (classification) — xác định đối tượng thuộc lớp nào, và *định vị* (localization) — xác định vị trí đối tượng trong ảnh thông qua bounding box.

2.1.1 Phân loại kiến trúc

Các kiến trúc Object Detection được chia thành hai nhóm chính:

- **Two-stage detectors** (ví dụ: R-CNN, Faster R-CNN): Giai đoạn 1 đề xuất các vùng ứng viên (Region Proposals), giai đoạn 2 phân loại và tinh chỉnh bounding box. Ưu điểm là độ chính xác cao, nhược điểm là tốc độ chậm.
- **One-stage detectors** (ví dụ: SSD, YOLO): Dự đoán bounding box và lớp đối tượng trong một lần lan truyền xuôi (single forward pass). Ưu điểm là tốc độ nhanh, phù hợp ứng dụng thời gian thực.

Ngoài ra, các phương pháp còn được phân loại theo cách xử lý anchor:

- **Anchor-based**: Sử dụng các hộp neo (anchor boxes) với kích thước và tỷ lệ được định nghĩa trước làm tham chiếu cho dự đoán.
- **Anchor-free**: Trực tiếp dự đoán tọa độ bounding box mà không cần anchor boxes, giúp đơn giản hóa kiến trúc và giảm siêu tham số cần tinh chỉnh.

2.1.2 Các chỉ số đánh giá

IoU (Intersection over Union): Đo mức độ trùng khớp giữa hộp dự đoán và hộp thực tế:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (2.1)$$

Precision và Recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (2.2)$$

trong đó TP là True Positive (dự đoán đúng), FP là False Positive (dự đoán sai), FN là False Negative (bỏ sót).

mAP (mean Average Precision): Trung bình của Average Precision trên tất cả các lớp:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (2.3)$$

Trong báo cáo này, hai chỉ số mAP chính được sử dụng:

- **mAP50:** Tính AP tại ngưỡng IoU ≥ 0.50 .
- **mAP50-95:** Trung bình AP tại các ngưỡng IoU từ 0.50 đến 0.95 (bước nhảy 0.05), phản ánh chính xác hơn chất lượng định vị.

2.2 Dòng YOLO

YOLO (*You Only Look Once*) là dòng mô hình one-stage object detection nổi bật nhất, đặc trưng bởi khả năng dự đoán bounding box và xác suất lớp chỉ trong một lần lan truyền xuôi duy nhất.

2.2.1 Lịch sử phát triển

Kể từ phiên bản đầu tiên (YOLOv1, 2015), dòng YOLO đã trải qua nhiều thế hệ cải tiến:

- **YOLOv1–v3:** Xây dựng nền tảng kiến trúc Darknet, giới thiệu multi-scale prediction.
- **YOLOv4–v5:** Tối ưu hóa kỹ thuật huấn luyện (CSP, Mosaic augmentation, PANet).
- **YOLOX:** Chuyển sang Decoupled Head (tách nhánh classification và regression), giới thiệu anchor-free detection.
- **YOLOv8:** Thống nhất kiến trúc cho nhiều tác vụ (detection, segmentation, pose), sử dụng DFL (Distribution Focal Loss) cho hồi quy bounding box.
- **YOLOv9:** Giới thiệu PGI (Programmable Gradient Information) và GELAN.
- **YOLO11 [5]:** Cải tiến về hiệu suất và hỗ trợ đa tác vụ.
- **YOLO26 [2]:** Loại bỏ NMS hoàn toàn (NMS-free), thay DFL bằng Direct Regression, giới thiệu ProgLoss và MuSGD optimizer — đây là mô hình được sử dụng trong đồ án.

2.2.2 Đặc điểm nổi bật của YOLO26

YOLO26 mang lại 3 cải tiến cốt lõi so với các phiên bản trước:

- 1) **NMS-Free:** Suy luận end-to-end hoàn toàn, loại bỏ bước hậu xử lý NMS giúp giảm độ trễ CPU tới 43%.
- 2) **Kiến trúc hợp nhất:** Hỗ trợ đồng thời Detection, Segmentation, Pose, OBB và Classification trong cùng một kiến trúc.

- 3) **MuSGD Optimizer**: Kết hợp SGD truyền thống và Muon (trực giao hóa gradient bằng Newton-Schulz) giúp hội tụ nhanh và ổn định hơn.

2.3 Các khái niệm nền tảng

2.3.1 Tích chập (Convolution)

Phép tích chập 2D là phép toán cơ bản trong mạng neural xử lý ảnh. Một lớp tích chập trượt bộ lọc (kernel) kích thước $k \times k$ trên ảnh đầu vào để tạo ra feature map:

$$y(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x(i+m, j+n) \cdot w(m, n) + b \quad (2.4)$$

Trong YOLO26, mỗi khối Conv bao gồm ba thành phần: $Conv2d \rightarrow BatchNorm \rightarrow SiLU activation$.

2.3.2 Batch Normalization (BN)

Chuẩn hóa đầu ra của mỗi lớp theo batch, giúp ổn định quá trình huấn luyện:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \cdot \gamma + \beta \quad (2.5)$$

2.3.3 Hàm kích hoạt SiLU

SiLU (Sigmoid Linear Unit), còn gọi là Swish, là hàm kích hoạt được sử dụng xuyên suốt trong YOLO26:

$$SiLU(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (2.6)$$

So với ReLU, SiLU cho phép gradient khác không ở vùng giá trị âm, giúp mô hình học tốt hơn.

2.3.4 Feature Pyramid Network (FPN) và PANet

FPN (Feature Pyramid Network): Tạo đặc trưng đa tỷ lệ bằng cách truyền thông tin từ các tầng sâu (high-level, semantic-rich) ngược lên các tầng nông (low-level, detail-rich) thông qua đường truyền top-down và phép cộng/ghép nối.

PANet (Path Aggregation Network): Bổ sung thêm đường truyền bottom-up, giúp thông tin chi tiết từ các tầng nông cũng được truyền xuống các tầng sâu, tăng cường khả năng phát hiện đối tượng ở mọi kích thước.

YOLO26 sử dụng kiến trúc Neck kết hợp cả FPN (top-down) và PAN (bottom-up) để tổng hợp đặc trưng đa tỷ lệ.

2.3.5 CIoU (Complete Intersection over Union)

CIoU mở rộng IoU bằng cách bổ sung thêm các yếu tố hình học:

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} - \alpha v \quad (2.7)$$

trong đó:

- $\rho(\mathbf{b}, \mathbf{b}^{gt})$: Khoảng cách Euclid giữa tâm hộp dự đoán và hộp thực tế.
- c : Đường chéo của hộp bao nhỏ nhất chứa cả hai hộp.
- $v = \frac{4}{\pi^2} (\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h})^2$: Hệ số đo sự khác biệt tỷ lệ khung hình.
- $\alpha = \frac{v}{(1-\text{IoU})+v}$: Trọng số cân bằng.

2.4 Edge Computing và Raspberry Pi 5

Edge Computing (điện toán biên) là mô hình xử lý dữ liệu ngay tại thiết bị đầu cuối thay vì gửi về máy chủ trung tâm (cloud). Ưu điểm bao gồm: giảm độ trễ truyền dữ liệu, bảo mật dữ liệu tốt hơn, và hoạt động độc lập khi mất kết nối mạng.

Raspberry Pi 5 là bo mạch nhúng phổ biến với cấu hình:

- CPU: Broadcom BCM2712 Quad-core ARM Cortex-A76 @ 2.4GHz
- RAM: 4GB / 8GB LPDDR4X
- Hỗ trợ camera MIPI CSI, USB 3.0, GPIO, PCIe
- Hệ điều hành: Raspberry Pi OS (Linux)

So với các giải pháp AI edge khác (NVIDIA Jetson, Google Coral), Raspberry Pi 5 có ưu điểm về chi phí thấp, cộng đồng hỗ trợ lớn, nhưng bị hạn chế về hiệu năng GPU/NPU. Đồ án này chứng minh rằng với kiến trúc mô hình phù hợp (YOLO26) và runtime tối ưu (NCNN), CPU thuần của Raspberry Pi 5 vẫn đủ khả năng chạy object detection thời gian thực.

2.5 NCNN Framework

NCNN [6] là framework suy luận mạng neural mã nguồn mở được phát triển bởi Tencent, tối ưu hóa đặc biệt cho các thiết bị di động và nhúng sử dụng kiến trúc ARM.

Các đặc điểm nổi bật:

- **Viết bằng C++**, không phụ thuộc thư viện bên ngoài (zero dependency).
- **Tối ưu NEON/SIMD** cho ARM: Tận dụng tối đa các chỉ thị vector hóa của CPU ARM.

- **Hỗ trợ đa định dạng:** Chuyển đổi từ ONNX, PyTorch, TensorFlow, v.v.
- **Nhẹ và nhanh:** Dung lượng thư viện nhỏ, thời gian khởi động nhanh, phù hợp cho các thiết bị có bộ nhớ hạn chế.

So với TensorFlow Lite (runtime được khóa trước sử dụng [1]), NCNN có lợi thế về tốc độ trên CPU ARM nhờ tối ưu hóa trực tiếp cho kiến trúc phần cứng, không cần trải qua quá trình lượng tử hóa INT8 phức tạp.

CHƯƠNG 3

Xây dựng bộ dữ liệu

3.1 Phân tích yêu cầu dataset

Bài toán nhận diện diễn giả trong môi trường giảng đường và hội trường đặt ra các yêu cầu cụ thể cho bộ dữ liệu:

- 1) **Lớp đối tượng:** Chỉ cần một lớp duy nhất — person (người). Trong phạm vi Đề án 1, hệ thống chưa cần phân biệt “diễn giả” với “khán giả”.
- 2) **Đa dạng tư thế:** Diễn giả có thể đứng, ngồi, di chuyển, quay lưng, hoặc bị che khuất một phần.
- 3) **Kích thước ảnh:** Tối ưu cho thiết bị biên — ảnh kích thước nhỏ (320×320 pixel) để giảm tải tính toán trên Raspberry Pi 5.
- 4) **Số lượng đủ lớn:** Cần hàng chục nghìn ảnh để mô hình học được đặc trưng tổng quát, tránh overfitting.
- 5) **Không quá nhiều đối tượng:** Trong bối cảnh hội trường, thường chỉ có 1-3 diễn giả. Các ảnh có quá nhiều người (crowd) không phù hợp.

3.2 Khảo sát và lựa chọn dataset

Nhóm đã khảo sát ba nguồn dữ liệu tiềm năng:

3.2.1 COCO 2017 (Common Objects in Context)

COCO [7] là bộ dữ liệu chuẩn (benchmark dataset) phổ biến nhất trong lĩnh vực Object Detection, chứa hơn 200.000 ảnh với 80 lớp đối tượng. Lớp person (ID: 0) chiếm số lượng lớn nhất trong COCO.

Ưu điểm: Đa dạng bối cảnh, đa dạng tư thế, annotation chất lượng cao, được cộng đồng kiểm chứng rộng rãi.

Nhược điểm: Chứa nhiều lớp không cần thiết, nhiều ảnh có quá nhiều người (crowd scene), cần lọc và tiền xử lý.

3.2.2 SCB-05 (Smart Classroom Behavior)

Dataset chuyên biệt cho bài toán hành vi trong lớp học, chứa các lớp như Teacher, Student.

Ưu điểm: Bối cảnh phù hợp (lớp học), có nhãn phân biệt giáo viên/học sinh.

Nhược điểm: Quy mô nhỏ hơn COCO, đa dạng tư thế hạn chế, annotation không đồng đều.

3.2.3 TED Talk (tự tạo)

Trích xuất khung hình từ video TED Talk và tự gán nhãn.

Ưu điểm: Bối cảnh sát với thực tế (diễn giả thuyết trình).

Nhược điểm: Tốn công sức gán nhãn thủ công, số lượng hạn chế, thiếu đa dạng.

3.2.4 Kết luận lựa chọn

Sau khi đánh giá, nhóm quyết định sử dụng **COCO 2017** làm nguồn dữ liệu chính, tiến hành lọc và tiền xử lý để tạo bộ dữ liệu **COCO Person 320** phù hợp với bài toán. SCB-05 được sử dụng như bộ dữ liệu thử nghiệm bổ sung.

3.3 Dataset COCO Person 320

3.3.1 Quy trình tiền xử lý

Toàn bộ quy trình tiền xử lý được hiện thực trong notebook `PrepairCOCO.ipynb`, chạy trên Google Colab:

Bước 1: Lọc lớp đối tượng: Chỉ giữ lại các annotation thuộc lớp person (ID: 0) từ tập COCO 2017.

Bước 2: Loại ảnh không có person: Loại bỏ các ảnh không chứa bất kỳ đối tượng person nào.

Bước 3: Loại ảnh quá nhiều người: Loại bỏ các ảnh có hơn 9 bounding box person, tránh các ảnh crowd scene không phù hợp với bối cảnh diễn giả.

Bước 4: Loại ảnh có quá nhiều box lớn: Loại bỏ các ảnh có nhiều bounding box chiếm diện tích quá lớn so với kích thước ảnh.

Bước 5: Resize ảnh: Resize toàn bộ ảnh về kích thước 320×320 pixel sử dụng phương pháp `cv2.INTER_AREA` (phù hợp khi thu nhỏ ảnh, tránh aliasing).

Bước 6: Chuyển đổi annotation: Chuyển đổi annotation từ định dạng COCO (xywh tuyệt đối) sang định dạng YOLO (class, cx, cy, w, h — chuẩn hóa về $[0, 1]$), đồng thời cập nhật tọa độ theo tỷ lệ resize.

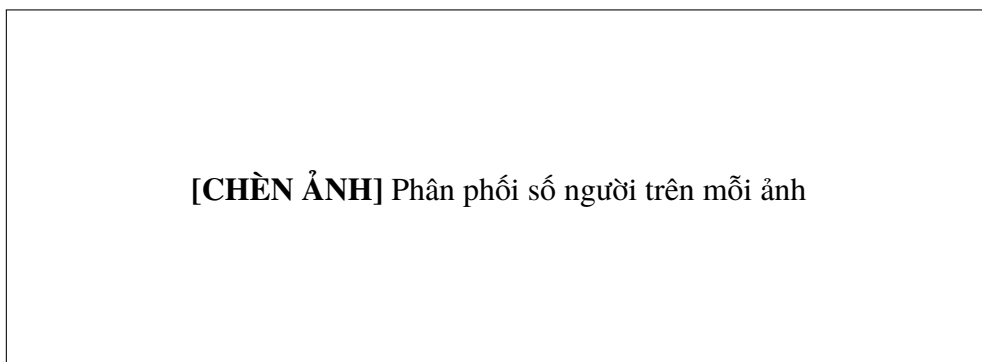
3.3.2 Thống kê dataset

Kết quả sau khi tiền xử lý được tóm tắt trong Bảng 3.1.

Bảng 3.1: Thống kê bộ dữ liệu COCO Person 320

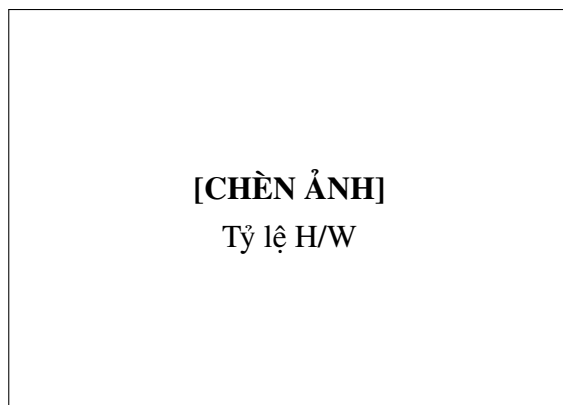
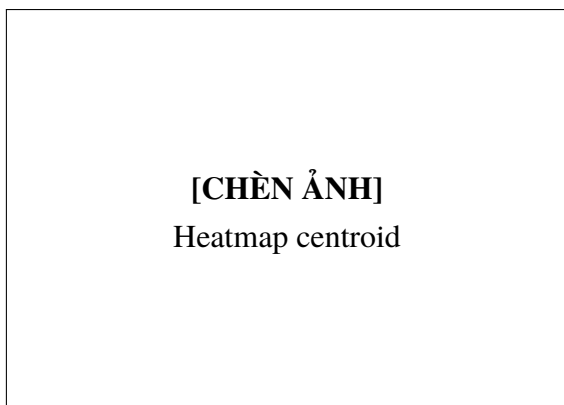
Tập dữ liệu	Số lượng ảnh	Tỷ lệ
Train	49.811	96%
Validation	2.074	4%
Tổng	51.885	100%

3.3.3 Phân tích dữ liệu khám phá (EDA)

**Hình 3.1:** Phân phối số lượng đối tượng person trên mỗi ảnh trong tập COCO Person 320

Kết quả phân tích EDA cho thấy:

- **Số lượng người/ảnh:** Phần lớn ảnh chứa 1–2 người (24.251 ảnh chứa 1 người, 10.433 ảnh chứa 2 người), phù hợp với bối cảnh diễn giả.
- **Tọa độ tâm:** Heatmap centroid cho thấy các bounding box tập trung ở vùng giữa ảnh ($x \approx 0.5$, $y \approx 0.5$), phản ánh xu hướng đặt đối tượng ở trung tâm khung hình.
- **Diện tích bbox:** Phân bố long-tail, đa phần bounding box có kích thước nhỏ đến trung bình.
- **Tỷ lệ H/W:** Peak ở khoảng ~ 2.0 (người đứng thường cao gấp đôi chiều rộng), phù hợp với đặc điểm hình học của cơ thể người.

**Hình 3.2:** (Trái) Heatmap tọa độ tâm bounding box. (Phải) Phân phối tỷ lệ cao/rộng của bounding box.

3.4 Định dạng YOLO

Mỗi ảnh trong dataset được đi kèm một file annotation .txt cùng tên, trong đó mỗi dòng tương ứng với một bounding box:

class_id cx cy w h

trong đó:

- class_id: ID lớp đối tượng (trong trường hợp này luôn là 0 — person).
- cx, cy: Tọa độ tâm bounding box, chuẩn hóa về $[0, 1]$ theo kích thước ảnh.
- w, h: Chiều rộng và chiều cao bounding box, chuẩn hóa về $[0, 1]$.

Ví dụ một dòng annotation: 0 0.4523 0.6218 0.1875 0.4063

3.5 Hướng mở rộng dataset

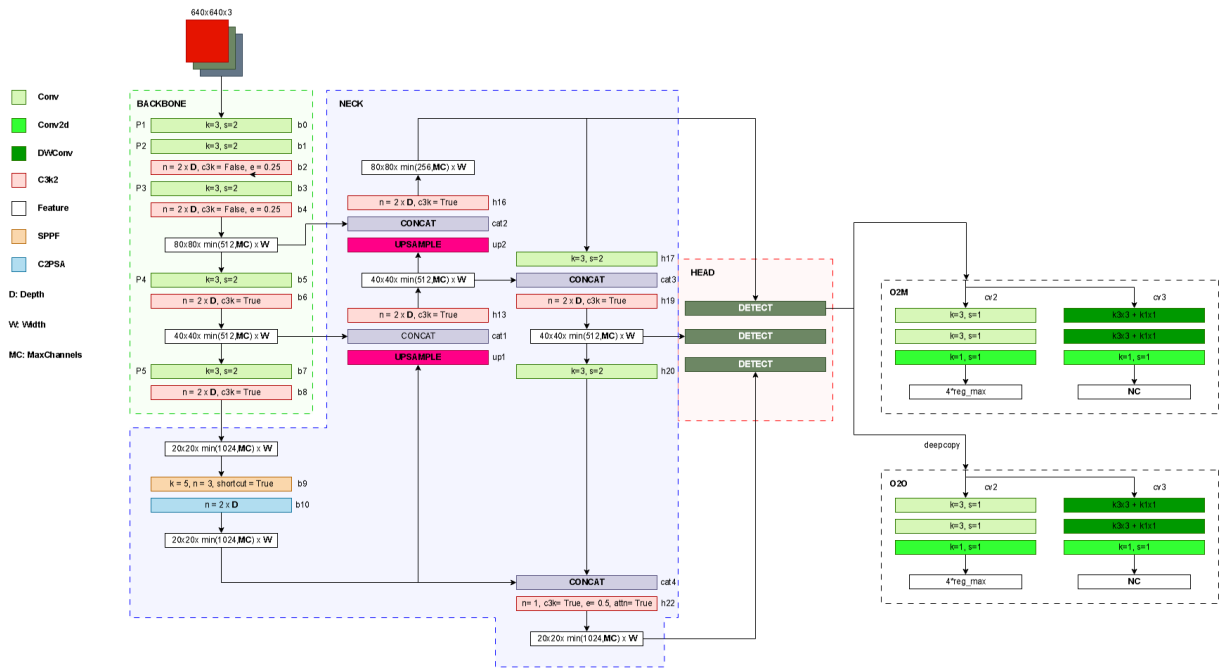
Trong các giai đoạn tiếp theo (Đồ án 2 và Khóa luận tốt nghiệp), nhóm dự kiến mở rộng dataset theo các hướng:

- **SCB-05**: Sử dụng dataset lớp học thông minh để bổ sung bối cảnh giảng dạy, đã thử nghiệm sơ bộ với 11.848 ảnh train / 4.699 ảnh validation.
- **TED Talk**: Trích xuất và gán nhãn khung hình từ các video TED Talk, tạo dataset chuyên biệt cho bối cảnh hội trường.
- **Dữ liệu thực tế**: Thu thập ảnh/video trực tiếp tại giảng đường và hội trường của trường để tạo dataset phù hợp nhất.

CHƯƠNG 4

Kiến trúc mô hình YOLO26

Chương này trình bày chi tiết kiến trúc mô hình YOLO26, bao gồm ba phần chính: Backbone (trích xuất đặc trưng), Neck (tổng hợp đặc trưng đa tỷ lệ), và Detection Head (dự đoán bounding box và lớp đối tượng). Nội dung được tổng hợp từ paper gốc [2], mã nguồn Ultralytics [4], và quá trình nghiên cứu xây dựng lại mô hình từ đầu (from scratch) của nhóm.



Hình 4.1: Sơ đồ khối tổng quát kiến trúc YOLO26. Đầu vào ảnh $3 \times 640 \times 640$ được xử lý qua Backbone (trích xuất đặc trưng), Neck (tổng hợp FPN+PAN), và Detection Head (Dual-Head O2M/O2O).

4.1 Backbone

Backbone của YOLO26 gồm 11 lớp (b0–b10) có nhiệm vụ trích xuất đặc trưng ảnh ở nhiều mức phân giải khác nhau.

4.1.1 Khối Conv cơ bản

Trong YOLO26, mỗi khối Conv bao gồm ba phần: lớp tích chập (Conv2d), chuẩn hóa batch (BatchNorm2d), và hàm kích hoạt SiLU:

```
1 class Conv(nn.Module):
2     def __init__(self, c1, c2, k=1, s=1, p=None, g=1):
3         self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p), groups=g,
4                                bias=False)
5         self.bn = nn.BatchNorm2d(c2)
```

```

5         self.act = nn.SiLU(inplace=True)
6
7     def forward(self, x):
8         return self.act(self.bn(self.conv(x)))

```

Listing 4.1: Khối Conv cơ bản trong YOLO26

Các lớp Conv đầu tiên (b0, b1, b3, b5, b7) sử dụng kernel 3×3 với stride $s = 2$ để giảm kích thước không gian xuống một nửa ở mỗi bước.

Conv 1x1 trộn channel tại cùng vị trí spatial. **Conv 3x3** trộn thông tin không gian (cạnh, biên, texture).

4.1.2 Khối DWConv (Depthwise Convolution)

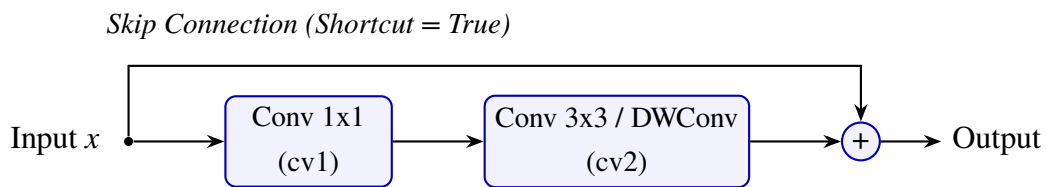
DWConv là tích chập tách biệt theo chiều sâu (Depthwise Convolution), trong đó số lượng nhóm tính toán bằng số kênh vào và số kênh ra ($\text{groups} = \text{gcd}(c_1, c_2)$):

$$\text{Params}_{\text{DW}} = c_1 \times k^2 \ll \text{Params}_{\text{std}} = c_1 \times c_2 \times k^2 \quad (4.1)$$

DWConv được sử dụng rộng rãi trong classification head (cv3) của YOLO26 nhằm giảm chi phí tính toán và tăng tốc độ suy luận.

4.1.3 Khối Bottleneck

Bottleneck là block residual cơ bản, tạo đặc trưng $F(x)$ bằng 2 lớp Conv rồi tùy điều kiện cộng residual với đầu vào x :

**Hình 4.2:** Sơ đồ khối Bottleneck với skip connection.

$$F(x) = \text{cv2}(\text{cv1}(x)) \quad (4.2)$$

$$\text{Output} = \begin{cases} x + F(x) & \text{nếu shortcut = True và } c_1 = c_2 \\ F(x) & \text{ngược lại} \end{cases} \quad (4.3)$$

trong đó:

- cv1: $\text{Conv}(c_1, c_-, k_1, s = 1)$ với $c_- = \lfloor c_2 \times e \rfloor$ (kênh ẩn, e là expansion ratio).
- cv2: $\text{Conv}(c_-, c_2, k_2, s = 1)$ với $\text{groups} = g$.
- Nhánh skip chỉ kích hoạt khi $c_1 = c_2$ và $\text{shortcut} = \text{True}$.

Tác dụng: Giảm chi phí tính toán (nhờ expansion ratio $e < 1$), cho phép xây dựng mạng sâu hơn mà vẫn nhẹ, và giữ thông tin gốc nhờ nhánh residual giúp tránh vanishing gradient.

Ví dụ: input $[B, 128, 80, 80]$, $e = 0.5 \rightarrow c_- = 64 \rightarrow$ output $[B, 128, 80, 80]$ (chiều dài và chiều rộng giữ nguyên).

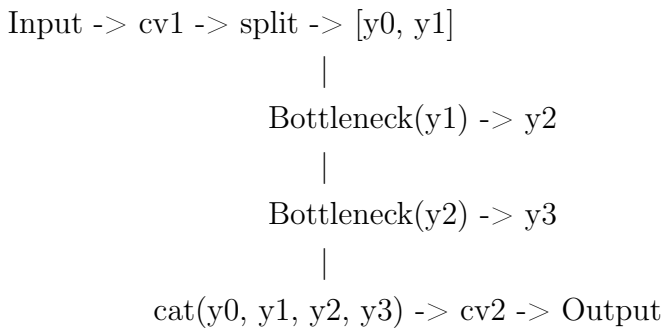
4.1.4 Khối C2f (Cross Stage Partial with 2 Convolutions)

Khối C2f [8] kết hợp tư tưởng của CSPNet nhằm gia tăng số lượng đường truyền gradient, giúp mạng học được nhiều đặc trưng đa dạng hơn:

$$C2f(x) = cv2\left(\text{Concat}\left[y_0, y_1, m_1(y_1), m_2(m_1(y_1)), \dots\right]\right) \quad (4.4)$$

Đầu vào sau khi qua cv1 được chia đôi thành 2 nhánh: $[y_0, y_1] = \text{chunk}(cv1(x), 2)$. Nhánh nông y_0 đi thẳng không xử lý sâu thêm, trong khi nhánh y_1 được truyền qua một chuỗi các khối Bottleneck m_i .

Sơ đồ luồng dữ liệu (với $n=2$):



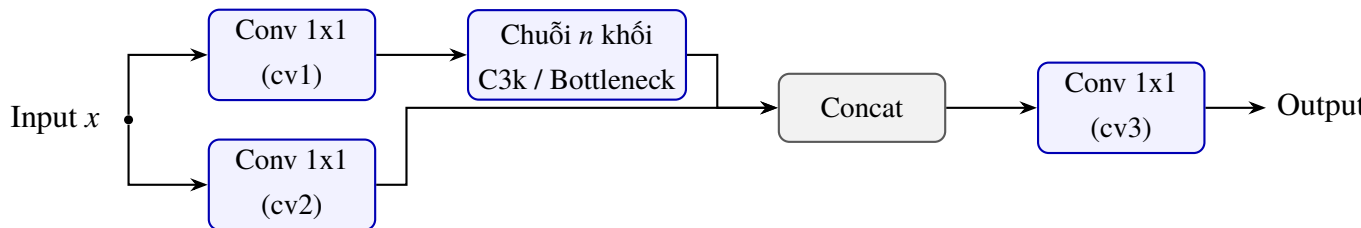
Ý nghĩa: Cơ chế split-and-concat này vừa giữ lại các đặc trưng nông thô (y_0) vừa trích xuất các đặc trưng ngữ cảnh sâu (y_3), giúp gradient lan truyền hiệu quả hơn trong quá trình backward.

4.1.5 Khối C3 và C3k

C3 là khối CSP kiểu cũ có hai nhánh chính: nhánh xử lý sâu qua chuỗi Bottleneck và nhánh đi thẳng qua một khối Conv, sau đó ghép lại bằng phép concat và xử lý qua Conv 1x1 cuối.

C3k kế thừa C3 nhưng sử dụng Bottleneck với kernel $k \times k$ tùy chỉnh (thường $k = 3$):

$$C3k : \text{Bottleneck}_{C3} \text{ dùng } k = ((1, 1), (3, 3)) \rightarrow C3k \text{ dùng } k = (3, 3) \quad (4.5)$$



Hình 4.3: Sơ đồ khối C3k — C3 với kernel 3x3 sâu hơn.

Khối C3k có khả năng bao quát thông tin không gian tốt hơn nhờ mở rộng kernel tích chập ở nhánh sâu, giúp học được các đặc trưng biên và cấu trúc hình học tốt hơn, đổi lại chi phí tính toán cao hơn Bottleneck thông thường.

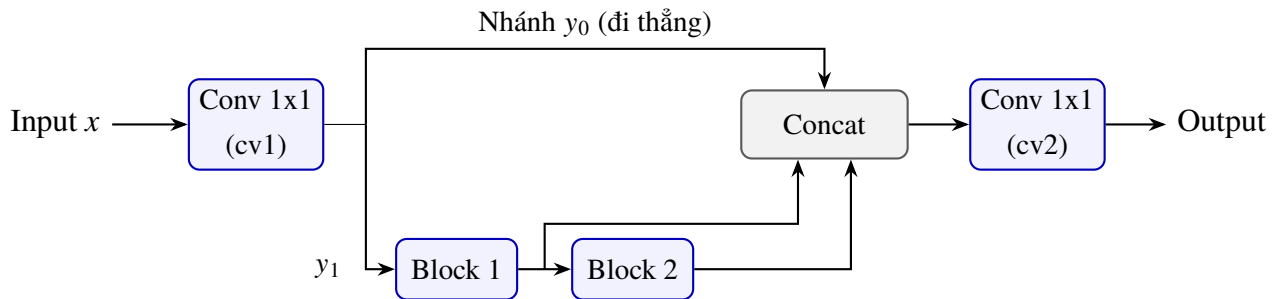
4.1.6 Khối C3k2

C3k2 là khối trộn đặc trưng chính và quan trọng nhất trong YOLO26, kế thừa từ C2f nhưng cho phép tùy biến loại block bên trong:

$$y_0, y_1 = \text{Split}(\text{cv1}(x)) \quad (4.6)$$

$$z_i = \text{Block}_i(z_{i-1}), \quad z_0 = y_1, \quad i = 1, \dots, n \quad (4.7)$$

$$\text{Output} = \text{cv2}(\text{cat}([y_0, y_1, z_1, \dots, z_n])) \quad (4.8)$$



Hình 4.4: Sơ đồ khối C3k2 — forward theo C2f, block bên trong tùy cấu hình.

Tùy vào 2 tham số boolean `attn` và `c3k`, các Block bên trong sẽ được khởi tạo khác nhau:

1. `attn=True`: Block = Bottleneck + PSABlock (Position-Sensitive Attention).
2. `attn=False, c3k=True`: Block = C3k (kernel $k \times k$ sâu).
3. `attn=False, c3k=False`: Block = Bottleneck tiêu chuẩn.

Bảng 4.1: Sử dụng C3k2 tại các vị trí trong YOLO26.

Layer	c3k	attn	Module bên trong	Vị trí
b2, b4	False	False	Bottleneck đơn giản	Backbone đầu
b6, b8	True	False	C3k (3x3 sâu)	Backbone P4, P5
h13–h19	True	False	C3k	Neck
h22	True	True	Bottleneck + PSABlock	Neck P5 output

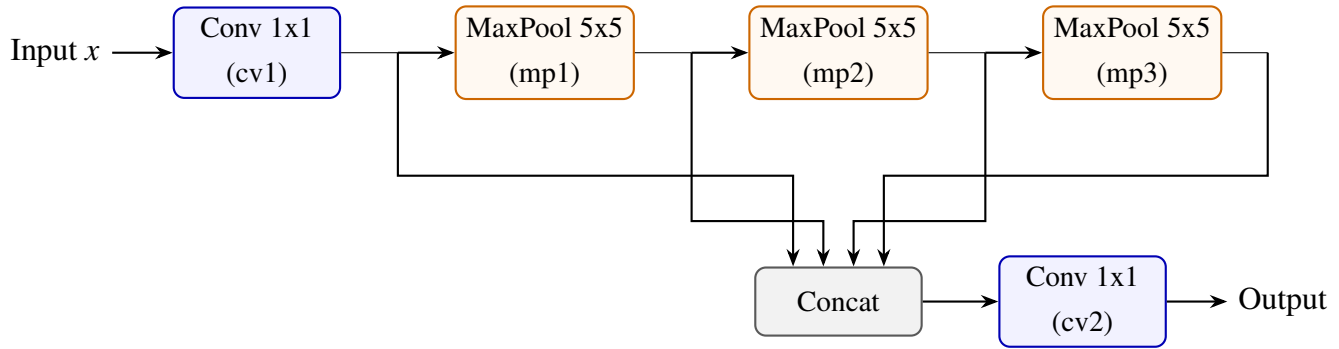
Ví dụ sự thay đổi kích thước tensor: $\text{C3k2}(c_1 = 1024, c_2 = 256, n = 2, c3k = \text{True}, e = 0.5)$:

$[B, 1024, 80, 80] \rightarrow cv1 \rightarrow [B, 256, 80, 80]$
 $\rightarrow \text{split: } y0 [B, 128, 80, 80], y1 [B, 128, 80, 80]$
 $\rightarrow C3k(y1) \rightarrow y2 [B, 128, 80, 80]$
 $\rightarrow C3k(y2) \rightarrow y3 [B, 128, 80, 80]$
 $\rightarrow \text{cat}(y0, y1, y2, y3) \rightarrow [B, 512, 80, 80]$
 $\rightarrow cv2 \rightarrow [B, 256, 80, 80]$

Kích thước không gian H, W được giữ nguyên hoàn toàn do tất cả các lớp tích chập bên trong đều sử dụng stride=1 và cơ chế tự động đệm (autopad).

4.1.7 Khối SPPF (Spatial Pyramid Pooling Fast)

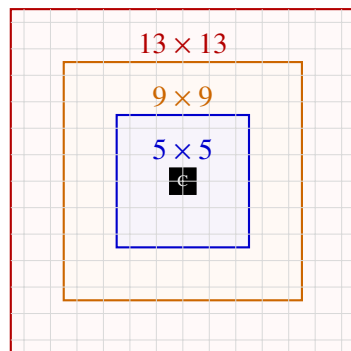
SPPF mở rộng vùng nhận cảm ngữ cảnh (receptive field) bằng cách áp dụng liên tiếp 3 lớp MaxPool 5×5 song song với việc lưu lại kết quả trung gian, mà không làm giảm kích thước không gian ảnh đầu vào:



Hình 4.5: Sơ đồ khối SPPF — MaxPool liên tiếp tạo multi-scale receptive field.

$$y_0 = cv1(x), \quad y_1 = \text{MaxPool}(y_0), \quad y_2 = \text{MaxPool}(y_1), \quad y_3 = \text{MaxPool}(y_2) \quad (4.9)$$

$$\text{Output} = cv2(\text{cat}([y_0, y_1, y_2, y_3])) \quad (4.10)$$



Lần 1 (mp1): Receptive Field 5×5

Lần 2 (mp2): Receptive Field 9×9

Lần 3 (mp3): Receptive Field 13×13

C: Điểm ảnh trung tâm (Center Pixel)

Hình 4.6: Minh họa sự mở rộng receptive field tương đương 5×5 , 9×9 , và 13×13 qua 3 lần MaxPool 5×5 liên tiếp.

Mỗi lần MaxPool 5×5 (với stride $s = 1$, padding $p = 2$) mở rộng vùng ngữ cảnh thêm 4 pixel. Qua 3 lần lặp, thông tin ngữ cảnh toàn cục được tổng hợp tương đương một cửa sổ MaxPool có kích thước 13×13 , hỗ trợ mô hình nhận diện vật thể ở nhiều kích thước khác nhau.

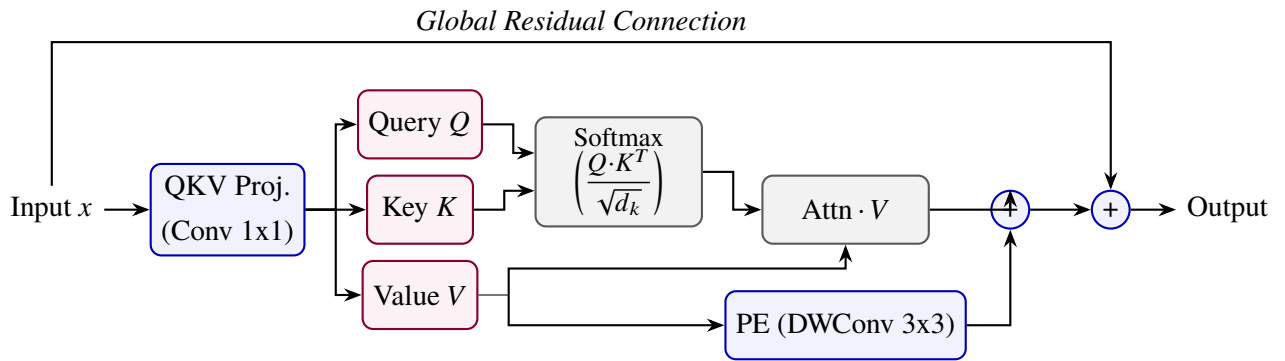
So với SPP: SPP dùng 3 kernel song song (5×5 , 9×9 , 13×13); trong khi SPPF dùng cùng một kernel 5×5 nối tiếp — cho receptive field tương đương nhưng **giảm đáng kể chi phí tính toán**.

Lưu ý: YOLO26 SPPF có act=False trên cv1

Khác với SPPF gốc (Ultralytics) dùng act=True (SiLU) trên lớp tích chập cv1, YOLO26 sử dụng act=False. Thiết kế này làm giảm dung lượng biểu diễn phi tuyến tính ngay trước bước max-pooling, giúp ổn định thông tin biên của vật thể lớn.

4.1.8 Cơ chế Attention (Multi-Head Self-Attention)

Trong các cấu hình nâng cao của YOLO26, Multi-Head Self-Attention được chèn vào Neck (h22) và Backbone (b10) để bắt cặp các tương quan không gian toàn cục:



Hình 4.7: Sơ đồ khối Attention với QKV projection và Position Encoding.

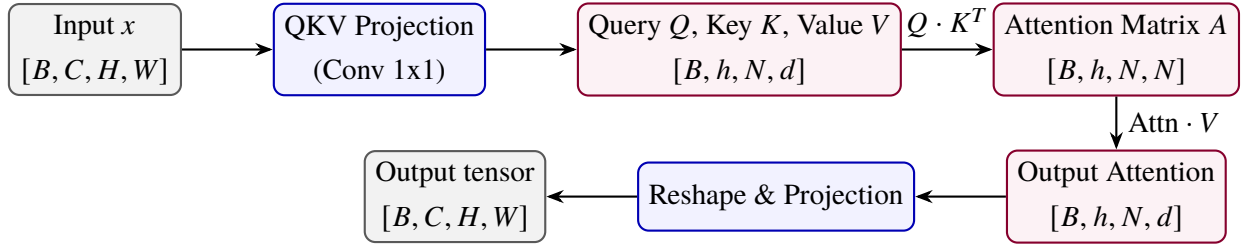
$$Q, K, V = \text{split}(\text{QKV_Conv}(x)) \quad (4.11)$$

$$\text{Attn} = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \quad (4.12)$$

$$\text{Output} = V \cdot \text{Attn}^T + \text{PE}(V) \quad (4.13)$$

- **QKV Conv:** Lớp tích chập Conv 1×1 chiếu đồng thời vector đầu vào sang 3 không gian Query (Q), Key (K), Value (V). Lớp này không sử dụng activation.
- $d_k = \text{head_dim} \times \text{attn_ratio}$: Kích thước vector Key được nén nhỏ để tiết kiệm bộ nhớ.
- **PE (Position Encoding):** Sử dụng một khối Depthwise Conv 3×3 tác động lên Value (V) nhằm bảo toàn thông tin cấu trúc không gian hình học vốn bị mất trong cơ chế tính toán ma trận Attention thông thường.

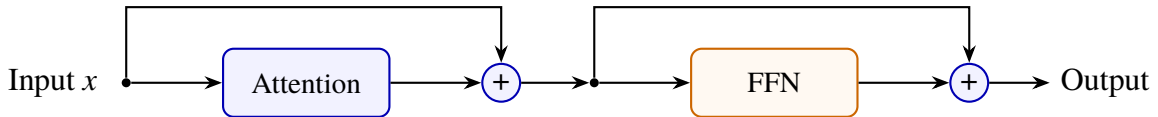
- **Multi-head:** Kênh được chia thành h heads riêng biệt để xử lý song song các vùng đặc trưng độc lập.



Hình 4.8: Chi tiết luồng xử lý và sự biến đổi kích thước tensor trong module Attention.

4.1.9 Khối PSABlock (Position-Sensitive Attention Block)

PSABlock tích hợp cơ chế Self-Attention bên trên và một khối Feed-Forward Network (FFN) theo dạng nối tiếp, đi kèm hai kết nối tắt (residual connection):



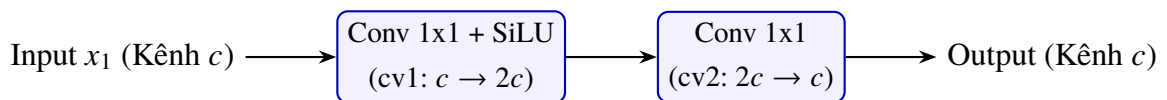
Hình 4.9: Sơ đồ PSABlock: Attention + FFN với các kết nối residual.

$$x_1 = x + \text{Attention}(x) \quad (\text{Residual Attention}) \quad (4.14)$$

$$x_2 = x_1 + \text{FFN}(x_1) \quad (\text{Residual FFN}) \quad (4.15)$$

FFN (Feed-Forward Network) gồm lớp Conv 1×1 tăng số kênh ($c \rightarrow 2c$), đi qua hàm kích hoạt SiLU, và lớp Conv 1×1 nén kênh ngược về ban đầu ($2c \rightarrow c$, act=False):

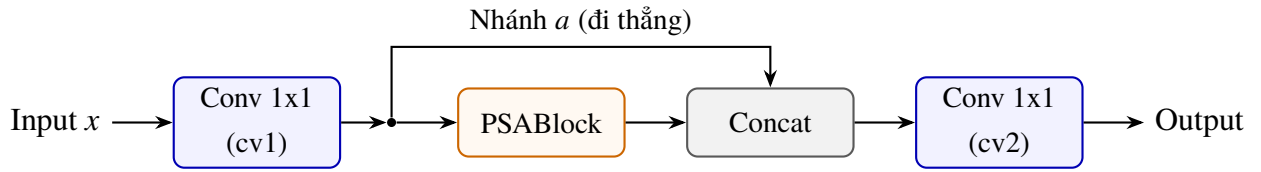
$$\text{FFN}(x_1) = \text{cv2}\left(\text{SiLU}\left(\text{cv1}(x_1)\right)\right) \quad (4.16)$$



Hình 4.10: Sơ đồ khối FFN (Feed-Forward Network) trong PSABlock.

4.1.10 Khối C2PSA (Channel-Split Position-Sensitive Attention)

Khối C2PSA áp dụng cơ chế chia nhánh (Channel-Split) của CSPNet kết hợp với PSABlock nhằm tối ưu hóa chi phí tính toán của cơ chế Self-Attention:



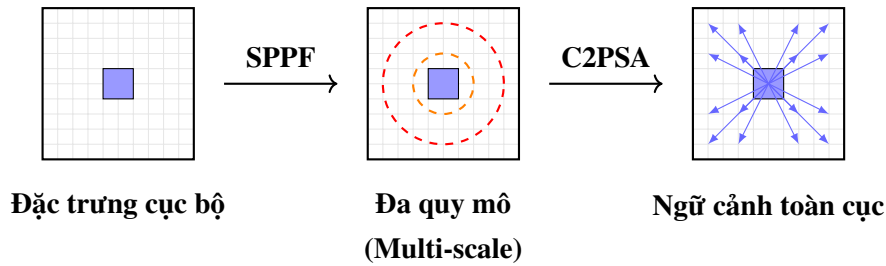
Hình 4.11: Sơ đồ C2PSA: CSP splitting + PSABlock.

$$a, b = \text{split}(\text{cv1}(x)) \quad (4.17)$$

$$b' = \text{PSABlock}(b) \quad (4.18)$$

$$\text{Output} = \text{cv2}(\text{Concat}[a, b']) \quad (4.19)$$

Đầu vào qua cv1 được phân đôi: nhánh a đi thẳng bảo toàn đặc trưng tích chập cục bộ, nhánh b đi qua PSABlock học các phụ thuộc không gian xa. Cuối cùng, hai nhánh được concat và hợp nhất qua cv2.



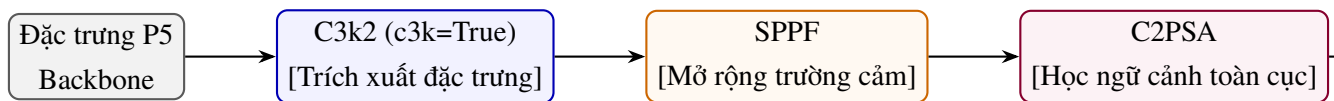
Hình 4.12: Mô phỏng sự biến đổi của feature map khi đi qua SPPF và C2PSA.

Tại sao chỉ dùng Attention ở P5?

Độ phức tạp tính toán của cơ chế Self-Attention là $O(N^2)$ với $N = H \times W$ là số điểm ảnh. Ở tầng P5 (10×10), $N = 100$ nên chi phí cực kỳ nhẹ. Ngược lại, ở tầng P3 (40×40), $N = 1600$ khiến độ phức tạp tăng gấp 256 lần, quá nặng cho các mô hình nhỏ như YOLO26-Nano.

4.1.11 Tổng quan đồ thị các khối đặc biệt

Để trực quan hóa sự sắp đặt các khối tùy biến, đồ thị dưới đây biểu diễn mối liên kết giữa C3k2, SPPF và C2PSA trong mô hình YOLO26:



Hình 4.13: Tổng quan cách tổ chức các khối đặc biệt: C3k2, SPPF, và C2PSA trong YOLO26.

4.1.12 Bảng chi tiết các lớp Backbone

Bảng 4.2: Chi tiết các lớp trong Backbone của YOLO26n (scale nano, ảnh 320×320).

Layer	Module	Input	Output (n)	Key Args	Note
b0	Conv	3×320^2	16×160^2	$k=3, s=2$	P1/2
b1	Conv	16×160^2	32×80^2	$k=3, s=2$	P2/4
b2	C3k2	32×80^2	64×80^2	$c3k=F, e=0.25$	
b3	Conv	64×80^2	64×40^2	$k=3, s=2$	P3/8
b4	C3k2	64×40^2	128×40^2	$c3k=F, e=0.25$	
b5	Conv	128×40^2	128×20^2	$k=3, s=2$	P4/16
b6	C3k2	128×20^2	128×20^2	$c3k=T$	
b7	Conv	128×20^2	256×10^2	$k=3, s=2$	P5/32
b8	C3k2	256×10^2	256×10^2	$c3k=T$	
b9	SPPF	256×10^2	256×10^2	$k=5, n=3$	
b10	C2PSA	256×10^2	256×10^2	attention	

4.2 Neck

Neck của YOLO26 sử dụng kiến trúc kết hợp FPN (top-down) và PAN (bottom-up) để tổng hợp đặc trưng đa tỷ lệ từ Backbone trước khi đưa vào Detection Head.

4.2.1 Top-Down Pathway (FPN)

Truyền semantic information từ tầng sâu (P5) xuống tầng nông (P3):

1. Upsample b10 (256×10^2) $\times 2 \rightarrow 256 \times 20^2$
2. Concat với b6 (128×20^2) $\rightarrow 384 \times 20^2$
3. C3k2 $\rightarrow$ h13: 128×20^2
4. Upsample h13 $\times 2 \rightarrow 128 \times 40^2$
5. Concat với b4 (128×40^2) $\rightarrow 256 \times 40^2$
6. C3k2 $\rightarrow$ h16: 64×40^2 (P3 output)

4.2.2 Bottom-Up Pathway (PAN)

Truyền detail information từ tầng nông (P3) lên tầng sâu (P5):

1. Conv stride-2: h16 (64×40^2) $\rightarrow 64 \times 20^2$
2. Concat với h13 (128×20^2) $\rightarrow 192 \times 20^2$

3. C3k2 $\rightarrow$ h19: 128×20^2 (P4 output)
4. Conv stride-2: h19 (128×20^2) $\rightarrow 128 \times 10^2$
5. Concat với b10 (256×10^2) $\rightarrow 384 \times 10^2$
6. C3k2(attn=True) $\rightarrow$ h22: 256×10^2 (P5 output)

Neck tạo ra 3 đầu ra ở 3 mức phân giải:

- **P3_out** (h16): 80×80 , stride 8 — phát hiện vật thể **nhỏ**.
- **P4_out** (h19): 40×40 , stride 16 — phát hiện vật thể **trung bình**.
- **P5_out** (h22): 20×20 , stride 32 — phát hiện vật thể **lớn**.

Bảng 4.3: Số anchor points tại mỗi scale detection (ảnh 320×320).

Scale	Stride	Feature Map	Anchor Points	Phù hợp
P3	8	40×40	1600	Vật nhỏ
P4	16	20×20	400	Vật vừa
P5	32	10×10	100	Vật lớn
Tổng			2100	

4.3 Detection Head: Dual-Head O2M/O2O

YOLO26 sử dụng kiến trúc Dual-Head với hai nhánh dự đoán song song:

4.3.1 Nhánh cv2 — Box Prediction

Dự đoán tọa độ bounding box thông qua 2 lớp Conv 3×3 và một lớp Conv 1×1 cuối:

```

1 self.cv2 = nn.ModuleList(
2     nn.Sequential(
3         Conv(x, c2, 3),           # Conv 3x3
4         Conv(c2, c2, 3),         # Conv 3x3
5         nn.Conv2d(c2, 4 * self.reg_max, 1) # Conv 1x1 -> 4*reg_max
6     ) for x in ch
7 )

```

Listing 4.2: Nhánh cv2 (Box) trong Detection Head

Đầu ra: $4 \times \text{reg_max}$ kênh. Với Direct Regression ($\text{reg_max} = 1$), đầu ra chỉ có 4 kênh tương ứng 4 khoảng cách LTRB (left, top, right, bottom).

4.3.2 Nhánh cv3 — Class Prediction

Dự đoán xác suất lớp đối tượng sử dụng Depthwise Conv để tối ưu tốc độ:

```

1 self.cv3 = nn.ModuleList(
2     nn.Sequential(
3         nn.Sequential(DWConv(x, x, 3), Conv(x, c3, 1)), # DWConv +
          Conv
4         nn.Sequential(DWConv(c3, c3, 3), Conv(c3, c3, 1)),
5         nn.Conv2d(c3, self.nc, 1) # Conv 1x1 -> nc classes
6     ) for x in ch
7 )

```

Listing 4.3: Nhánh cv3 (Class) trong Detection Head

4.3.3 Cơ chế Dual-Head

Bảng 4.4: So sánh nhánh O2M và O2O trong Dual-Head

Đặc tính	O2M (One-to-Many)	O2O (One-to-One)
Khi nào dùng	Chỉ khi huấn luyện	Khi suy luận (và cả huấn luyện)
TAL topk	topk = 10	topk ₂ = 1
Số dự đoán / vật thể	Nhiều anchor → 1 vật thể	Đúng 1 anchor → 1 vật thể
Mục đích	Tạo gradient dày đặc, đẩy nhanh tốc độ học	NMS-free, dự đoán sạch
Cần NMS?	Có (khi suy luận đơn lẻ)	Không

Khi deploy thực tế, nhánh O2M bị loại bỏ hoàn toàn thông qua hàm `strip_o2m()`, chỉ giữ lại nhánh O2O phục vụ suy luận trực tiếp.

Bảng 4.5: So sánh model trước và sau khi strip O2M.

Metric	Training (last.pt)	Inference (lite.pt)	Giảm
File size	38.2 MB	5.6 MB	~85%
Precision	FP32	FP16	50%
O2M Head	Có	Stripped	—
Optimizer state	Có	Stripped	—

4.4 Box Decoding: Direct Regression

YOLO26 sử dụng phương pháp hồi quy trực tiếp (Direct Regression) thay cho DFL [9], giúp giảm đáng kể khối lượng tính toán:

Bước 1: Feature Extraction: Trích xuất đặc trưng qua Backbone và Neck.

Bước 2: Tạo Anchor Grids: Xác định tọa độ tâm (c_x, c_y) cho mỗi ô lưới.

Bước 3: Dự đoán LTRB: Nhánh cv2 xuất ra 4 giá trị khoảng cách: $d_{\text{left}}, d_{\text{top}}, d_{\text{right}}, d_{\text{bottom}}$.

Bước 4: Khôi phục tọa độ:

$$x_{\min} = c_x - d_{\text{left}}, \quad y_{\min} = c_y - d_{\text{top}} \quad (4.20)$$

$$x_{\max} = c_x + d_{\text{right}}, \quad y_{\max} = c_y + d_{\text{bottom}} \quad (4.21)$$

Bước 5: Rescale: Nhân tọa độ với stride tương ứng để đưa về kích thước ảnh gốc.

4.4.1 Cơ chế DFL (Distribution Focal Loss)

DFL [9] chuyển bài toán hồi quy khoảng cách liên tục thành bài toán phân loại trên một số lượng `reg_max` các bins rời rạc:

$$\hat{d} = \sum_{i=0}^{\text{reg_max}-1} i \cdot p_i, \quad p_i = \text{softmax}(\text{logits})_i \quad (4.22)$$

Trọng số của lớp Conv 1×1 cuối được cố định là chuỗi tăng dần $[0, 1, 2, \dots, \text{reg_max} - 1]$ và không tham gia vào quá trình lan truyền ngược để cập nhật tham số.

YOLO26 mặc định: `reg_max=1` — không dùng DFL

Khi đặt cấu hình mặc định `reg_max=1`, DFL sẽ tự động được thay thế bằng một lớp đồng nhất `nn.Identity()`. Lúc này mô hình thực hiện hồi quy trực tiếp 4 giá trị khoảng cách (l, t, r, b) , đầu ra của box head chỉ gồm 4 kênh. Với cấu hình `reg_max=16`, đầu ra sẽ có 64 kênh, cho phép dự đoán phân phối xác suất biên tốt hơn nhưng tăng chi phí tính toán.

4.4.2 So sánh Direct Regression và DFL

Bảng 4.6: So sánh hồi quy trực tiếp (Direct Regression) và DFL

Đặc tính	Direct Regression	DFL
Cách tính	Xuất trực tiếp 4 giá trị l, r, t, b	Softmax 16 giá trị/hướng, tổng trọng số
Kênh đầu ra	$4 \times 1 = 4$	$4 \times 16 = 64$
Ưu điểm	Tốc độ nhanh, đơn giản	Chính xác hơn ở biên mờ
Nhược điểm	Kém chính xác ở biên mờ	Softmax tốn CPU, gây nghẽn

YOLO26 bù đắp sự suy giảm độ chính xác của Direct Regression bằng cách đặt `topk = 10` ở nhánh O2M, tạo gradient dày đặc ép Backbone và Neck học đặc trưng sắc nét hơn.

4.5 Phân tỷ lệ mô hình (Model Scaling)

YOLO26 hỗ trợ nhiều biến thể kích thước thông qua 3 hệ số:

Bảng 4.7: Cấu hình Scale của YOLO26.

Scale	Depth	Width	Max Ch	Ghi chú
n (nano)	0.50	0.25	1024	Nhỏ nhất, nhanh nhất
s (small)	0.50	0.50	1024	Cân bằng tốc độ/chính xác
m (medium)	0.50	1.00	512	Trung bình
l (large)	1.00	1.00	512	Lớn
x (xlarge)	1.00	1.50	512	Chính xác nhất

Công thức tính kênh và lặp:

$$\text{ch}(c) = \text{make_divisible}(\min(c, \text{max_ch}) \times \text{width}, 8) \quad (4.23)$$

$$\text{rep}(n) = \max(\text{round}(n \times \text{depth}), 1) \quad (4.24)$$

Ví dụ scale nano: $\text{ch}(256) = \text{make_divisible}(\min(256, 1024) \times 0.25, 8) = 64$.

Trong đồ án, nhóm sử dụng biến thể **nano** (n) với tổng dung lượng weights khoảng 5.3MB, phù hợp cho triển khai trên Raspberry Pi 5.

CHƯƠNG 5

Huấn luyện và tối ưu mô hình

5.1 Huấn luyện YOLO26 gốc (Ultralytics)

5.1.1 Thiết lập

Phiên bản đầu tiên sử dụng thư viện Ultralytics [4] để huấn luyện YOLO26 trên dataset COCO Person 320:

```
1 from ultralytics import YOLO
2
3 model = YOLO("yolo26n.pt") # Pretrained nano
4 model.train(
5     data="coco_person_320.yaml",
6     epochs=120,
7     imgsz=224,
8     batch=64,
9     project="TRACKING_YOLO26",
10    name="yolo26_224"
11 )
```

Listing 5.1: Huấn luyện YOLO26 gốc với Ultralytics

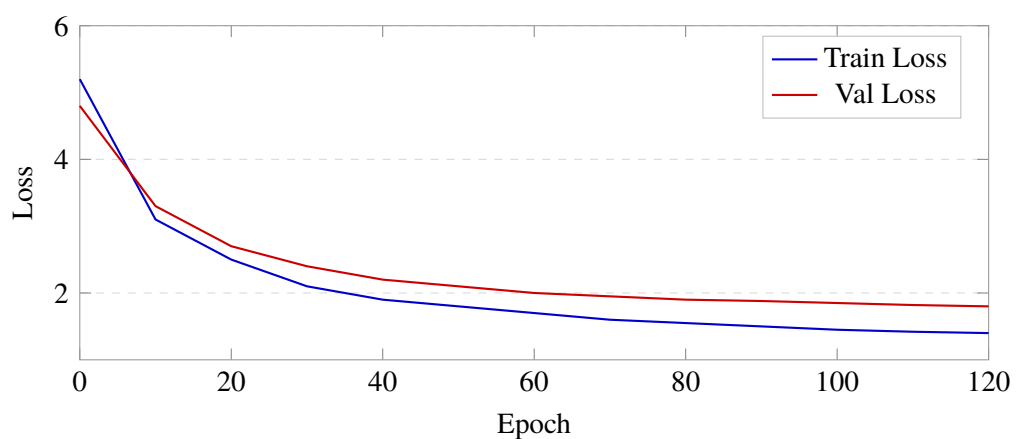
Bảng 5.1: Siêu tham số huấn luyện YOLO26 gốc

Siêu tham số	Giá trị
Nền tảng	Kaggle (GPU)
Mô hình cơ sở	yolo26n.pt (pretrained)
Epochs	120
Batch size	64
Image size	224×224
Learning rate (lr0)	0.01
LR final (lrf)	0.01
Momentum	0.937
Weight decay	0.0005
Warmup epochs	3.0
Box loss weight	7.5
Cls loss weight	0.5
DFL loss weight	1.5
Mosaic	1.0
Degrees (rotation)	5.0

5.1.2 Kết quả

Bảng 5.2: Kết quả huấn luyện YOLO26 gốc (epoch 120)

Chỉ số	Giá trị
mAP50	0.756
mAP50-95	0.532
Precision	0.840
Recall	0.645

**Hình 5.1:** Biểu đồ loss curves trong quá trình huấn luyện YOLO26 gốc qua 120 epochs.

		Predicted Label	
		Person	Background
True Label	Person	0.84	0.16
	Background	0.20	0.80

Hình 5.2: Ma trận nhầm lẫn (Confusion Matrix) của YOLO26 gốc trên tập validation.

5.2 Custom YOLO26 from scratch

5.2.1 Động lực

Mục đích của việc xây dựng lại YOLO26 từ đầu (from scratch) là:

- 1) **Hiểu sâu kiến trúc:** Nắm bắt chi tiết từng khối, từng phép toán trong mô hình thay vì chỉ sử dụng API có sẵn.
- 2) **Bổ trợ cho quá trình custom:** Tạo nền tảng để tùy biến mô hình (thay đổi scale, reg_max, loss function, optimizer) trong các giai đoạn tiếp theo.

Mã nguồn custom gồm 3 file chính:

- yolo26_modules.py: Kiến trúc mô hình (Backbone, Neck, Head, inference pipeline).
- yolo26_loss.py: Các hàm loss (E2ELoss, BboxLoss, TaskAlignedAssigner).
- train26.py: Training loop, data augmentation, optimizer setup.

5.2.2 Các tùy biến so với gốc

Bảng 5.3: So sánh cấu hình YOLO26 gốc (Ultralytics) và custom

Thành phần	Gốc (Ultralytics)	Custom
Framework	Ultralytics API	PyTorch thuần (from scratch)
reg_max	1 (direct regression)	1 (giữ nguyên)
Optimizer	MuSGD	MuSGD (tự hiện thực)
ProgLoss	Có	Có (tự hiện thực)
STAL	Có	Có (tự hiện thực)
Scale	Nano (n)	Nano (n) + custom u/t

5.3 Hàm mất mát (Loss Functions)

Tổng loss gồm 3 thành phần:

$$\mathcal{L}_{\text{head}} = w_{\text{box}} \cdot \mathcal{L}_{\text{box}} + w_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} + w_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}} \tag{5.1}$$

Bảng 5.4: Trọng số các thành phần loss (thay đổi theo ProgLoss).

Thành phần	Source	w đầu	w cuối	Mục đích
$\mathcal{L}_{\text{box}}$	cv2 $\rightarrow$ CIoU	7.5	3.75	Định vị (localization)
$\mathcal{L}_{\text{cls}}$	cv3 $\rightarrow$ BCE	0.5	1.5	Phân loại (classification)
$\mathcal{L}_{\text{dfl}}$	cv2 $\rightarrow$ DFL/L1	1.5	1.5	Phân phối khoảng cách

Box Loss — CIoU Loss: Xét cả khoảng cách tâm, diện tích giao, và tỷ lệ khung hình — gradient tốt hơn IoU thuần:

$$\mathcal{L}_{\text{box}} = \frac{\sum_{i \in \text{fg}} (1 - \text{CIoU}_i) \cdot w_i}{\sum_j t_j} \quad (5.2)$$

Classification Loss — BCE Loss: Target scores $t_{i,c}$ là **soft labels** từ alignment metric (không phải 0/1 cứng):

$$\mathcal{L}_{\text{cls}} = \frac{\sum_{i,c} \text{BCE}(\hat{s}_{i,c}, t_{i,c})}{\sum_j t_j} \quad (5.3)$$

LTRB Loss (khi `reg_max = 1`, mặc định YOLO26):

$$\mathcal{L}_{\text{L1}} = \frac{\sum_{i \in \text{fg}} \|p_i - t_i\|_1 \cdot w_i}{\sum_j t_j} \quad (5.4)$$

5.4 TaskAlignedAssigner (TAL)

TAL xác định anchor nào là positive/negative cho mỗi GT box dựa trên alignment metric:

$$\text{align_metric}_{i,j} = s_{i,j}^\alpha \cdot u_{i,j}^\beta \quad (5.5)$$

trong đó $s_{i,j}$ là classification score, $u_{i,j}$ là CIoU, $\alpha = 0.5$, $\beta = 6.0$ (CIoU được nhấn mạnh rất mạnh).

Thuật toán 1 TaskAlignedAssigner

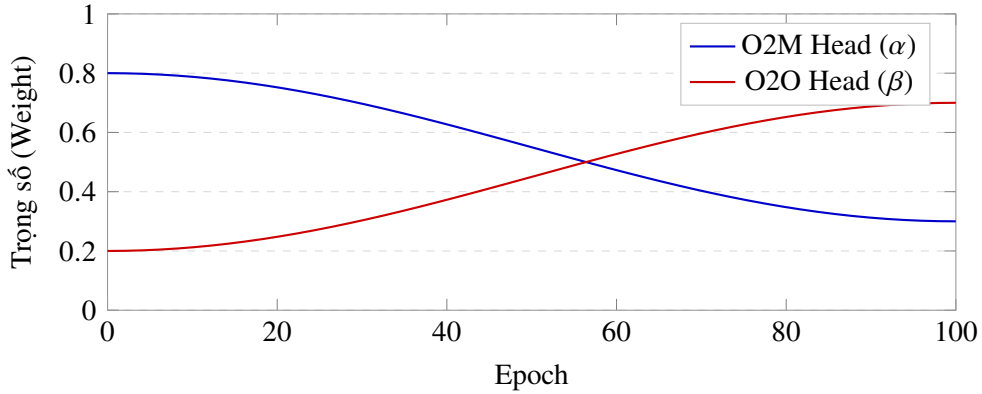
- 1: **Input:** `pred_scores`, `pred_bboxes`, `anchor_points`, `gt_labels`, `gt_bboxes`
 - 2: **Step 1:** Lọc anchors nằm trong GT box (geometric prior)
 - 3: **Step 2:** Tính `align_metric = s0.5 · CIoU6`
 - 4: **Step 3:** Chọn Top-K anchors có metric cao nhất cho mỗi GT
O2M: $K = 10$, O2O: $K = 7 \rightarrow$ chọn Top-1
 - 5: **Step 4:** Xử lý conflict: nếu anchor match nhiều GT, giữ GT có IoU cao nhất
 - 6: **Step 5:** Tính `soft_labels = one-hot × normalized metric`
 - 7: **Output:** `target_labels`, `target_bboxes`, `target_scores`, `fg_mask`
-

5.5 ProgLoss (Progressive Loss Balancing)

ProgLoss tự động điều chỉnh trọng số đóng góp của hai nhánh O2M và O2O theo tiến trình huấn luyện, tuân theo quy luật Cosine Decay:

$$\alpha(\text{epoch}) = \alpha_{\text{start}} + (\alpha_{\text{end}} - \alpha_{\text{start}}) \cdot \frac{1 - \cos(\pi \cdot \text{progress})}{2} \quad (5.6)$$

$$\beta = 1 - \alpha \quad (5.7)$$



Hình 5.3: Biểu đồ thể hiện sự biến đổi tỷ lệ của hai nhánh O2M (α) và O2O (β) theo tiến trình huấn luyện (ProgLoss).

Bảng 5.5: Lịch trình trọng số O2M/O2O qua epoch.

Epoch	α (O2M)	β (O2O)	Ý nghĩa
1	0.80	0.20	O2M ưu thế — dense supervision
50	0.55	0.45	Cân bằng dần
100	0.30	0.70	O2O tiếp quản — chuyên cho inference

Đồng thời, trọng số các thành phần loss cũng thay đổi:

- Box loss weight: $7.5 \rightarrow 3.75$ (giảm dần, tinh chỉnh mịn hơn).
- Cls loss weight: $0.5 \rightarrow 1.5$ (tăng dần, tập trung phân loại).
- DFL loss weight: giữ không đổi ở 1.5.

Triết lý: Giai đoạn đầu tập trung học ở đầu (box weight cao), giai đoạn sau tập trung học là gì (cls weight cao).

5.5.1 Công thức Loss tổng hợp đầy đủ

$$\mathcal{L}_{\text{total}}(e) = \alpha(e) \cdot \left[w_{\text{box}}(e) \cdot \mathcal{L}_{\text{box}}^{\text{O2M}} + w_{\text{cls}}(e) \cdot \mathcal{L}_{\text{cls}}^{\text{O2M}} + w_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}}^{\text{O2M}} \right] + \beta(e) \cdot \left[w_{\text{box}}(e) \cdot \mathcal{L}_{\text{box}}^{\text{O2O}} + w_{\text{cls}}(e) \cdot \mathcal{L}_{\text{cls}}^{\text{O2O}} + w_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}}^{\text{O2O}} \right] \quad (5.8)$$

Thực giá:

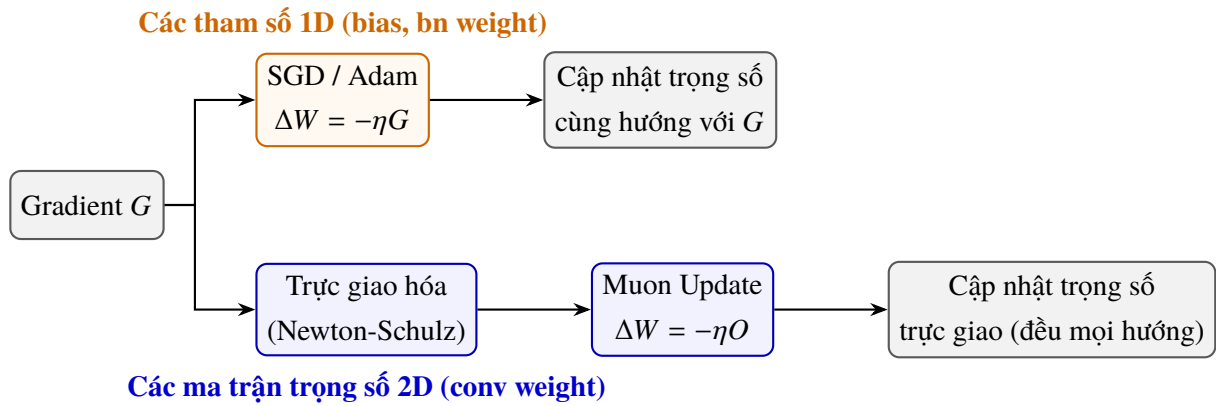
- Đầu training: α cao $\rightarrow$ nhiều anchor positive (dense supervision); w_{box} cao $\rightarrow$ ưu tiên localization.
- Cuối training: β cao $\rightarrow$ O2O head quan trọng hơn; w_{cls} cao $\rightarrow$ tập trung classification.

5.6 MuSGD Optimizer

MuSGD là bộ tối ưu lai kết hợp SGD và Muon [10]:

- **SGD:** Xử lý các tham số 1D (bias, gain, BatchNorm γ/β).
- **Muon:** Xử lý các ma trận trọng số 2D lớn (weights của Conv2d) bằng phép trực giao hóa gradient.

Nguyên lý Muon: Biến đổi ma trận gradient G thành ma trận cập nhật trực giao $O = UV^T$ (từ phân tích SVD $G = U\Sigma V^T$, loại bỏ Σ). Điều này giúp tất cả các chiều không gian được đối xử công bằng, tránh tình trạng gradient zig-zag ở các chiều dốc.



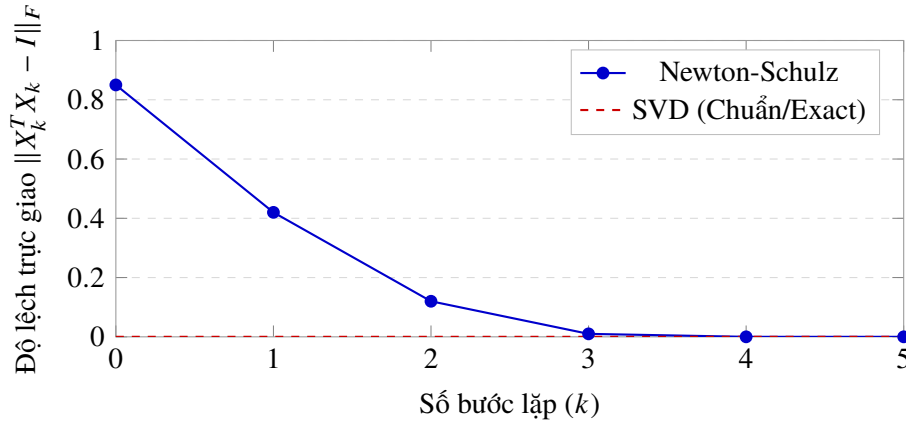
Hình 5.4: So sánh quy trình cập nhật trọng số giữa SGD và Muon trong bộ tối ưu MuSGD.

Newton-Schulz thay SVD: Trong thực tế, Muon không sử dụng SVD chính xác (quá chậm trên GPU) mà thay bằng thuật toán lặp Newton-Schulz:

$$X_0 = \frac{G}{\|G\|_F + \epsilon} \quad (5.9)$$

$$X_{k+1} = a \cdot X_k + b \cdot (X_k X_k^T) X_k + c \cdot (X_k X_k^T)^2 X_k \quad (5.10)$$

với các hệ số $a = 3.4445$, $b = -4.7750$, $c = 2.0315$, chạy 5 bước lặp. Kết quả đạt ma trận xấp xỉ trực giao với sai số $\kappa \approx 1.01$ so với $\kappa = 1.0$ lý tưởng, nhưng nhanh hơn SVD từ 10–100 lần trên GPU.



Hình 5.5: Tốc độ hội tụ trực giao hóa của thuật toán lặp Newton-Schulz so với SVD.

Ví dụ phân tích SVD từng bước của gradient G

Cho ma trận gradient $G = \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix}$ thu được từ quá trình lan truyền ngược. Mục tiêu của bộ tối ưu Muon là tìm ma trận cập nhật trực giao $O = UV^T$.

Bước 1: Tính ma trận hiệp phương sai $M = G^T G$:

$$M = \begin{bmatrix} 2 & 2 \\ -1 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 8 & 0 \\ 0 & 2 \end{bmatrix}$$

Do M đã là ma trận đường chéo, các trị riêng có thể thấy ngay lập tức.

Bước 2: Tìm các trị riêng λ của M : Giải phương trình đặc trưng $\det(M - \lambda I) = 0$:

$$(8 - \lambda)(2 - \lambda) = 0 \implies \lambda_1 = 8, \quad \lambda_2 = 2$$

Bước 3: Tìm các giá trị riêng lẻ (Singular Values) $\sigma_i = \sqrt{\lambda_i}$:

$$\sigma_1 = \sqrt{8} = 2\sqrt{2} \approx 2.8284, \quad \sigma_2 = \sqrt{2} \approx 1.4142$$

Ma trận các giá trị riêng lẻ Σ và số điều kiện κ là:

$$\Sigma = \begin{bmatrix} 2.83 & 0.00 \\ 0.00 & 1.41 \end{bmatrix}, \quad \kappa = \frac{2.83}{1.41} = 2.0$$

Bước 4: Tìm các vectơ riêng $(M - \lambda_i I)v_i = 0$ để xây dựng ma trận V :

- Với $\lambda_1 = 8$: $\begin{bmatrix} 0 & 0 \\ 0 & -6 \end{bmatrix} \begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies -6v_{12} = 0 \implies v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
- Với $\lambda_2 = 2$: $\begin{bmatrix} 6 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_{21} \\ v_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies 6v_{21} = 0 \implies v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

Do đó, $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$ (Ma trận đơn vị).

Bước 5: Tìm ma trận U từ liên hệ $u_i = \frac{1}{\sigma_i} G v_i$:

$$\bullet u_1 = \frac{1}{2.8284} \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{2.83} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$$

$$\bullet u_2 = \frac{1}{1.4142} \begin{bmatrix} 2 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{1.41} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.7071 \\ 0.7071 \end{bmatrix}$$

Do đó, $U = \begin{bmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{bmatrix}$.

Bước 6: Khôi phục ma trận trực giao hóa $O = UV^T$:

$$O = \begin{bmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.7071 & -0.7071 \\ 0.7071 & 0.7071 \end{bmatrix}$$

Ma trận cập nhật O là một ma trận trực giao (tất cả các giá trị riêng lẻ của nó đều bằng 1.0).

Tại sao chuẩn hóa bằng chuẩn Frobenius?

Để thuật toán lặp Newton-Schulz hội tụ về ma trận trực giao, điều kiện tiên quyết là các giá trị riêng lẻ σ_i của ma trận khởi tạo X_0 phải nằm trong khoảng $(0, \sqrt{3})$.

Khi khởi tạo $X_0 = \frac{G}{\|G\|_F}$, ta có:

$$\|X_0\|_F^2 = \sum_i \sigma_i(X_0)^2 = 1 \implies \sigma_i(X_0) \leq 1 < \sqrt{3}$$

Điều này đảm bảo mọi giá trị riêng lẻ của ma trận khởi tạo đều nhỏ hơn 1 (nằm sâu trong vùng hội tụ). Nếu không chuẩn hóa bằng chuẩn Frobenius $\|G\|_F$, một số giá trị riêng lẻ cực lớn của gradient G có thể vượt quá $\sqrt{3}$, làm cho chuỗi lặp Newton-Schulz bị phân kỳ.

5.7 STAL (Small-Target-Aware Label Assignment)

STAL giải quyết thách thức phát hiện đối tượng kích thước cực nhỏ:

$$\text{Label Assignment} = \begin{cases} \text{Ép gán 4 anchors} & \text{nếu kích thước vật thể} < 8 \times 8 \text{ pixel} \\ \text{TAL tiêu chuẩn (IoU + Cls)} & \text{ngược lại} \end{cases} \quad (5.11)$$

Việc ép buộc gán tối thiểu 4 anchor cho vật thể nhỏ đảm bảo chúng luôn đóng góp đáng kể vào hàm loss, buộc mô hình phải học cách nhận diện.

5.8 Hàm strip_o2m()

Khi chuyển từ huấn luyện sang triển khai:

```
1 def strip_o2m(model):
2     """Loại bỏ hoàn toàn nhanh O2M khi deploy."""
3     for m in model.modules():
4         if hasattr(m, 'one2many'):
5             del m.one2many
6     # Chỉ giữ lại nhanh O2O -> inference NMS-free
```

Listing 5.2: Loại bỏ nhánh O2M khi deploy

5.9 Kết quả Custom (1st run)

Bảng 5.6: Kết quả huấn luyện YOLO26 Custom — lần chạy 1 (79 epochs)

Chỉ số	Giá trị
Platform	Kaggle T4 GPU
Epochs đã chạy	79
mAP50	0.686
Box loss (train)	1.166
Cls loss (train)	4.097
Val box loss	0.984
Val cls loss	4.143
Weights (best.pt)	~30.7 MB

Nhận xét:

- Custom model đạt mAP50 = 0.686 sau 79 epochs, so với gốc 0.756 sau 120 epochs. Mô hình vẫn đang trong quá trình hội tụ.
- Cls loss tăng cao ở cuối training (từ ~3.4 → 4.1) — đây là **dấu hiệu bình thường** của ProgLoss đang chuyển trọng số từ O2M sang O2O, ép cls loss của O2O tăng lên.

- Val box loss tiếp tục giảm tốt ($2.22 \rightarrow 0.98$), cho thấy mô hình đang học định vị hiệu quả.
- Weights lớn hơn nhiều so với gốc (~ 30.7 MB vs ~ 5.3 MB) do chứa cả O2M head và optimizer state. Sau khi `strip_o2m()`, kích thước sẽ giảm đáng kể.

5.10 Export pipeline

Quy trình chuyển đổi mô hình để triển khai trên Raspberry Pi 5:

$$\text{PyTorch (.pt)} \xrightarrow{\text{strip_o2m()}} \text{Cleaned .pt} \xrightarrow{\text{export}} \text{NCNN}$$

So với pipeline của khóa trước [1] (PyTorch $\rightarrow$ ONNX $\rightarrow$ TF $\rightarrow$ TFLite INT8 $\rightarrow$ Edge TPU), pipeline NCNN đơn giản hơn đáng kể chỉ với 2 bước, không cần lượng tử hóa INT8.

CHƯƠNG 6

Triển khai trên Raspberry Pi 5

6.1 Phần cứng

Bảng 6.1: Cấu hình phần cứng hệ thống

Thành phần	Chi tiết
Bo mạch	Raspberry Pi 5 Model B
CPU	Broadcom BCM2712, Quad-core ARM Cortex-A76 @ 2.4GHz
RAM	4GB LPDDR4X
AI Accelerator	Không có (không sử dụng Google Coral, Hailo, hay bất kỳ NPU nào)
Hệ điều hành	Raspberry Pi OS (Linux)

Điểm đặc biệt của đồ án này là hệ thống được triển khai trên Raspberry Pi 5 với cấu hình **tối thiểu** — chỉ 4GB RAM và **không có bộ tăng tốc AI chuyên dụng**. So với nghiên cứu tiền nhiệm [1] sử dụng RPi 5 8GB kết hợp USB Google Coral TPU, hệ thống của nhóm giảm đáng kể chi phí và độ phức tạp phần cứng.

6.2 Phần mềm và kết nối

- **Runtime:** NCNN [6] — framework inference tối ưu cho ARM, viết bằng C++, tận dụng NEON/SIMD.
- **Kết nối:** SSH từ laptop Windows qua WiFi. Trong trường hợp không cùng mạng LAN, sử dụng Tailscale để tạo VPN peer-to-peer.
- **Điều kiện benchmark:** Tắt hoàn toàn GUI (headless), chỉ chạy detection pipeline.

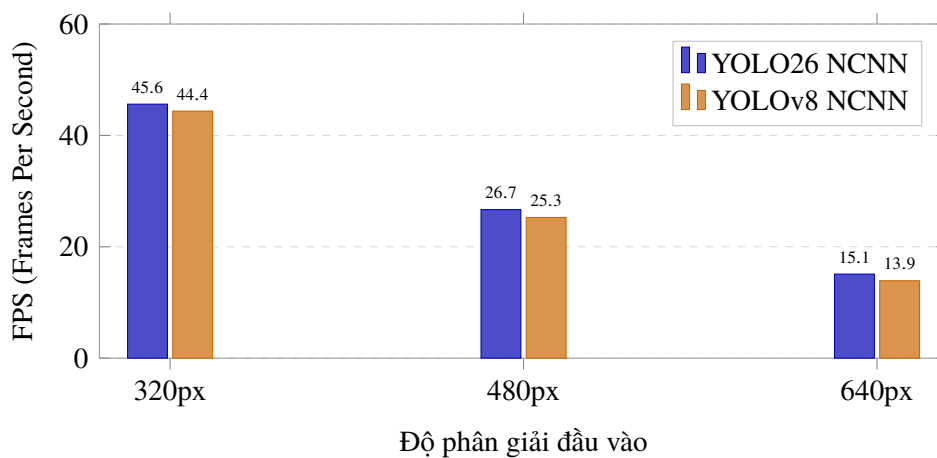
6.3 Benchmark FPS

Kết quả đo FPS trên Raspberry Pi 5 4GB, định dạng NCNN, chế độ detection-only (tắt GUI):

Bảng 6.2: Benchmark FPS trên Raspberry Pi 5 (NCNN, detection-only, tắt GUI)

Model	Resolution	Avg FPS
YOLO26 NCNN	320	45.61*
YOLO26 NCNN	480	26.67
YOLO26 NCNN	640	15.09
YOLOv8 NCNN	320	44.36
YOLOv8 NCNN	480	25.26
YOLOv8 NCNN	640	13.90

* Con số 45.61 FPS cần được xác minh lại (re-verify) trong các thí nghiệm tiếp theo.

**Hình 6.1:** So sánh FPS giữa YOLO26 NCNN và YOLOv8 NCNN trên Raspberry Pi 5 ở các độ phân giải đầu vào 320, 480, 640.**Nhận xét:**

- YOLO26 nhanh hơn YOLOv8 ở mọi độ phân giải, tuy nhiên mức chênh lệch không quá lớn (do cùng scale nano, kiến trúc nền tương đồng).
- FPS 45+ ở 320px đạt được nhờ 3 yếu tố: optimize đầu vào về 320×320 , tắt GUI hoàn toàn, và NCNN runtime tối ưu cho ARM.
- Ở 640px, FPS giảm xuống ~ 15 — vẫn đủ cho nhiều ứng dụng nhưng chưa đạt mức real-time mượt mà.

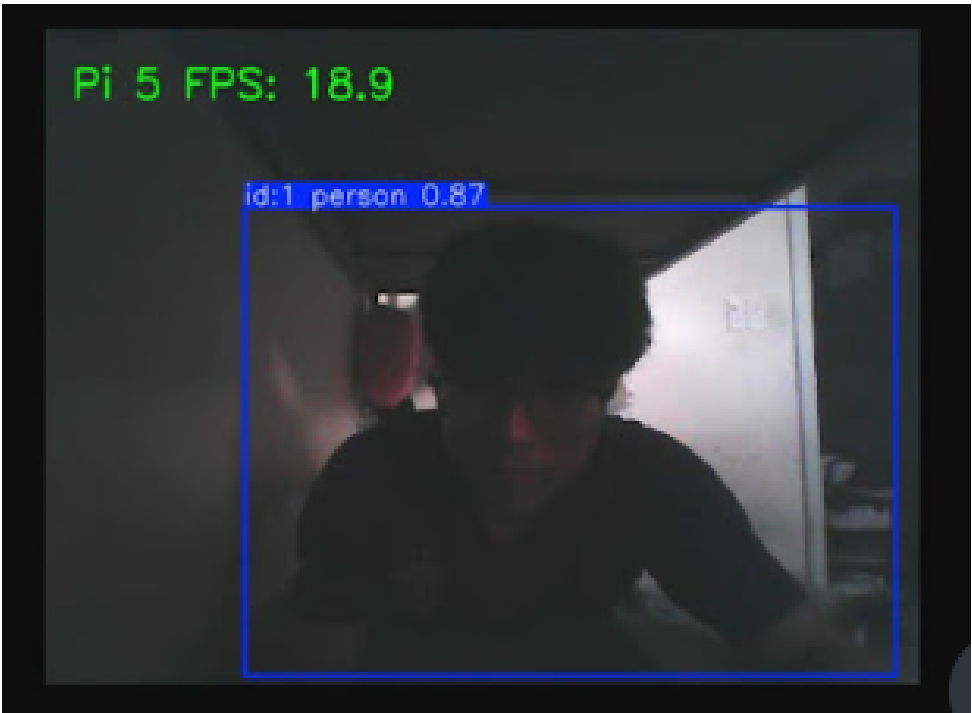
6.4 So sánh với nghiên cứu tiền nhiệm

Bảng 6.3: So sánh hệ thống triển khai với nghiên cứu tiền nhiệm [1]

Tiêu chí	Khóa trước (2025)	Nhóm hiện tại (2026)
RPi model	RPi 5, 8GB	RPi 5, 4GB
Accelerator	USB Google Coral TPU	Không có
Model	YOLOv8_n / YOLOv9_t	YOLO26n
Format	TFLite INT8	NCNN
Input	320×320	320×320
Inference	~15ms (V8_n, Coral)	cần re-verify
Export pipeline	.pt→ONNX→TF→TFLite→EdgeTPU	.pt→ NCNN
Chi phí accelerator	~\$60	\$0

Nhóm tiền nhiệm sử dụng RPi 5 8GB kết hợp USB Google Coral TPU, đạt inference time ~15ms/frame (YOLOv8_n, TFLite INT8, 320×320). Nhóm hiện tại đạt hiệu năng detection tương đương trên CPU thuần của RPi 5 4GB mà không cần accelerator, nhờ kết hợp YOLO26 (NMS-free, Direct Regression) với NCNN runtime tối ưu cho ARM. Điều này giảm chi phí phần cứng và đơn giản hóa pipeline triển khai.

6.5 Demo



Hình 6.2: Demo nhận diện người trên Raspberry Pi 5 với camera 1280×720, input resolution 640, đạt 18.9 FPS. Detection: id:1 person 0.87.

CHƯƠNG 7

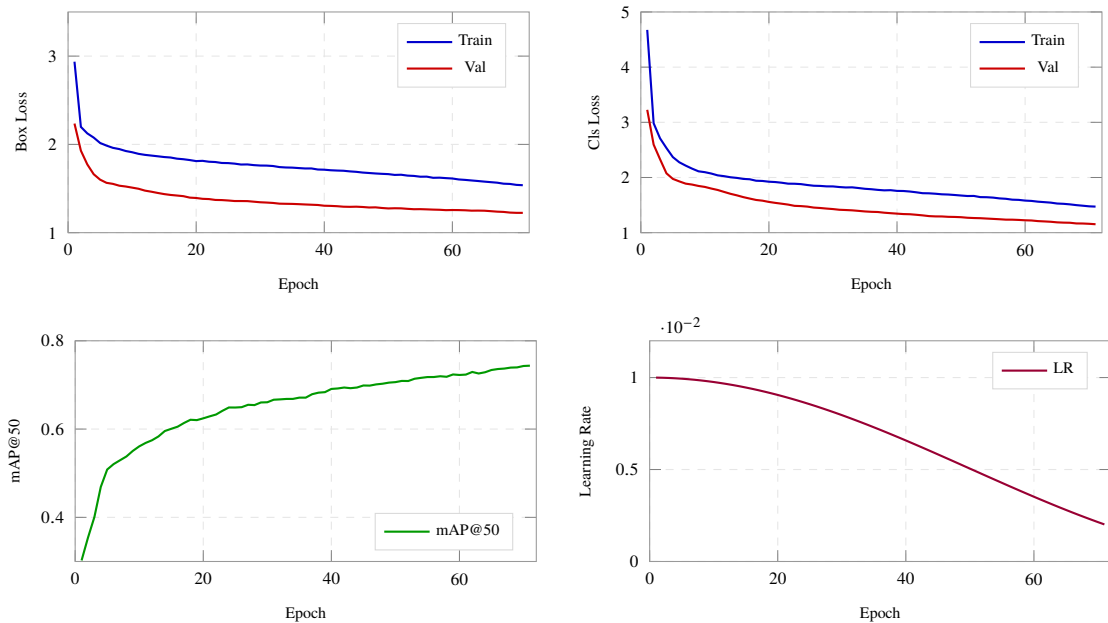
Kết quả và đánh giá

7.1 Tổng hợp kết quả huấn luyện

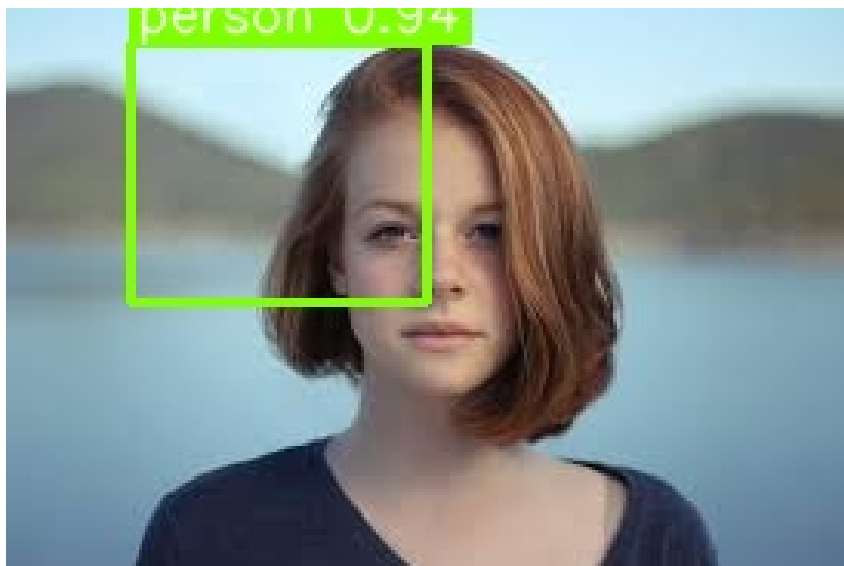
Bảng 7.1: Tổng hợp kết quả huấn luyện các phiên bản YOLO26

Phiên bản	Epochs	mAP50	mAP50-95
YOLO26 gốc (Ultralytics)	120	0.756	0.532
YOLO26 Custom (1st run)	79	0.686	—

Phiên bản Ultralytics đạt kết quả tốt hơn nhờ được huấn luyện đầy đủ 120 epochs với pretrained weights. Phiên bản custom vẫn đang trong quá trình hội tụ sau 79 epochs và dự kiến sẽ tiếp tục cải thiện.



Hình 7.1: Biểu đồ kết quả huấn luyện và so sánh mAP của mô hình YOLO26 Custom qua các epochs (trực quan hóa từ dữ liệu results.csv).



Hình 7.2: Hình ảnh mô phỏng kết quả nhận diện thực tế (prediction samples) của mô hình YOLO26.

7.2 Kết quả inference trên Raspberry Pi 5

Kết quả sơ bộ cho thấy YOLO26 NCNN đạt tốc độ detection-only ~ 45 FPS¹ ở độ phân giải 320×320 trên Raspberry Pi 5 4GB mà không cần accelerator.

7.3 Đánh giá

7.3.1 Thành tựu

- 1) **Nghiên cứu kiến trúc:** Nhóm đã nghiên cứu và trình bày chi tiết toàn bộ kiến trúc YOLO26, từ Backbone đến Detection Head, bao gồm cả các cơ chế huấn luyện tiên tiến (ProgLoss, MuSGD, STAL).
- 2) **Custom implementation:** Xây dựng thành công YOLO26 từ đầu (from scratch) bằng PyTorch thuần, đạt $mAP50 = 0.686$ sau 79 epochs — chứng minh sự hiểu biết sâu về kiến trúc.
- 3) **Triển khai edge:** Demo thành công detection trên Raspberry Pi 5 4GB không cần accelerator, với pipeline triển khai đơn giản (NCNN).
- 4) **Dataset:** Xây dựng bộ dữ liệu COCO Person 320 gồm 49.811 ảnh huấn luyện, lớn hơn đáng kể so với khóa trước (2.207 ảnh).

7.3.2 Hạn chế

- 1) Chưa tích hợp thuật toán bám vết (tracking) — hệ thống mới dừng ở mức detection.

¹Con số cần được xác minh lại trong các thí nghiệm tiếp theo.

- 2) Chưa có hệ thống điều khiển servo Pan-Tilt.
- 3) Custom model chưa hoàn tất huấn luyện đầy đủ (79/120+ epochs).
- 4) Chưa demo custom model trên Raspberry Pi 5.
- 5) Cần xác minh lại con số FPS benchmark.

7.4 Hướng phát triển

7.4.1 Đồ án 2 (dự kiến)

- Tích hợp ByteTrack [3] để bám vết diễn giả liên tục.
- Hoàn thiện custom YOLO26 và triển khai trên Raspberry Pi 5.
- Thiết kế hệ thống điều khiển servo Pan-Tilt cơ bản.

7.4.2 Khóa luận tốt nghiệp (dự kiến)

- Hoàn thiện toàn bộ pipeline: detection → tracking → PID servo control.
- Xây dựng giao diện giám sát và điều khiển.
- Truyền dẫn video thời gian thực qua mạng.
- Tối ưu hóa mô hình cho các điều kiện ánh sáng và bối cảnh đa dạng.
- Thu thập dataset thực tế tại giảng đường UIT.

PHỤ LỤC A

Phụ lục: Code nguồn chính

A.1 Kiến trúc YOLO26n

```
1 def build_yolo26n(nc=80):
2     d, w, mc = 0.50, 0.25, 1024
3     def ch(c): return max(round(min(c, mc) * w), 1) & ~7
4     def rep(n): return max(round(n * d), 1)
5
6     backbone = nn.ModuleList([
7         Conv(3, ch(64), 3, 2), # 0 P1/2
8         Conv(ch(64), ch(128), 3, 2), # 1 P2/4
9         C3k2(ch(128), ch(256), rep(2), False, 0.25),
10        Conv(ch(256), ch(256), 3, 2), # 3 P3/8
11        C3k2(ch(256), ch(512), rep(2), False, 0.25),
12        Conv(ch(512), ch(512), 3, 2), # 5 P4/16
13        C3k2(ch(512), ch(512), rep(2), True),
14        Conv(ch(512), ch(1024), 3, 2), # 7 P5/32
15        C3k2(ch(1024), ch(1024), rep(2), True),
16        SPPF(ch(1024), ch(1024), 5, 3),
17        C2PSA(ch(1024), ch(1024), rep(2)),
18    ])
19    detect = Detect(nc=nc, reg_max=1, end2end=True,
20                  ch=(ch(256), ch(512), ch(1024)))
21    return backbone, head, detect
```

Listing A.1: build_yolo26n — Xây dựng model YOLO26 scale nano

A.2 make_anchors

```
1 def make_anchors(feats, strides, grid_cell_offset=0.5):
2     anchor_points, stride_tensor = [], []
3     for feat, stride in zip(feats, strides):
4         _, _, h, w = feat.shape
5         sx = torch.arange(w, device=feat.device) + grid_cell_offset
6         sy = torch.arange(h, device=feat.device) + grid_cell_offset
7         sy, sx = torch.meshgrid(sy, sx, indexing='ij')
8         anchor_points.append(torch.stack([sx, sy], -1).view(-1, 2))
```



```

9         stride_tensor.append(torch.full((h*w, 1), stride, device=feat
        .device))
10     return torch.cat(anchor_points), torch.cat(stride_tensor)

```

Listing A.2: make_anchors — Tạo anchor points cho detection

A.3 dist2bbox và bbox2dist

```

1 def dist2bbox(distance, anchor_points, xywh=True, dim=-1):
2     lt, rb = distance.chunk(2, dim)
3     x1y1 = anchor_points - lt
4     x2y2 = anchor_points + rb
5     if xywh:
6         c_xy = (x1y1 + x2y2) / 2
7         wh = x2y2 - x1y1
8         return torch.cat([c_xy, wh], dim)
9     return torch.cat([x1y1, x2y2], dim)
10
11 def bbox2dist(anchor_points, bbox, reg_max=None):
12     x1y1, x2y2 = bbox.chunk(2, -1)
13     dist = torch.cat((anchor_points - x1y1, x2y2 - anchor_points),
14                       -1)
15     if reg_max is not None:
16         dist = dist.clamp_(0, reg_max - 0.01)
17     return dist

```

Listing A.3: dist2bbox — Chuyển LTRB distances thành bounding box

A.4 Newton–Schulz Iteration

```

1 def newton_schulz_orthogonalize(G, steps=5):
2     # Chuan hoa Frobenius
3     X = G / G.norm() # ||X_0||_F = 1
4
5     # He so cubic convergence
6     a, b, c = 3.4445, -4.7750, 2.0315
7
8     for _ in range(steps):
9         A = X @ X.T
10        X = a * X + b * (A @ X) + c * (A @ (A @ X))
11
12    return X # X ~ U V^T

```

Listing A.4: Newton–Schulz orthogonalization trong Muon optimizer

A.5 E2ELoss decay

```

1 class E2ELoss:
2     def __init__(self, model):
3         self.o2m = 0.8
4         self.o2o = 0.2
5         self.final_o2m = 0.3
6
7     def decay(self, x):
8         return (max(1 - x / max(epochs - 1, 1), 0)
9                 * (self.o2m_copy - self.final_o2m)
10                + self.final_o2m)
11
12    def update(self):
13        self.updates += 1
14        self.o2m = self.decay(self.updates)
15        self.o2o = max(1.0 - self.o2m, 0)
16        # ProgLoss: box/cls weight update
17        progress = min(self.updates / max(self.prog_epochs - 1, 1),
18                        1.0)
19        self.one2many.box_weight = 7.5 + (3.75 - 7.5) * progress
20        self.one2many.cls_weight = 0.5 + (1.5 - 0.5) * progress

```

Listing A.5: E2ELoss — Progressive decay O2M/O2O weights

Tài liệu tham khảo

- [1] B. T. Kiệt **and** M. H. G. Bảo, ?Hiện thực thiết bị camera tự động theo dõi diễn giả trong buổi hội thảo, thuyết trình,? GVHD: ThS. Trần Hoàng Lộc, Khóa luận tốt nghiệp, Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM, 2025.
- [2] G. Jocher **and** J. Qiu, ?Yolo26: Redesigning yolo for end-to-end and efficient detection,? *arXiv preprint arXiv:2502.15737*, 2025. **url:** <https://arxiv.org/abs/2502.15737>
- [3] Y. Zhang **and others**, ?Bytetrack: Multi-object tracking by associating every detection box,? *European Conference on Computer Vision (ECCV)*, 2022.
- [4] G. Jocher, *Ultralytics yolo*, <https://github.com/ultralytics/ultralytics>, 2023.
- [5] G. Jocher **and** J. Qiu, ?Ultralytics yolo11,? *arXiv preprint arXiv:2412.18230*, 2024. **url:** <https://arxiv.org/abs/2412.18230>
- [6] nihui, *Ncnn: A high-performance neural network inference framework*, <https://github.com/Tencent/ncnn>, Tencent, 2017.
- [7] T.-Y. Lin, M. Maire, S. Belongie **and others**, *Microsoft coco: Common objects in context*, <https://cocodataset.org>, 2014.
- [8] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh **and** I.-H. Yeh, ?Cspnet: A new backbone that can enhance learning capability of cnn,? *IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2020.
- [9] X. Li **and others**, ?Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,? *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [10] K. Jordan, ?Muon: An optimizer for hidden layers in neural networks,? 2024. **url:** <https://kellerjordan.github.io/posts/muon/>